

Notice

This course was originally recorded using **Brackets** as a code editor. However, as it's been discontinued by Adobe, we will be covering here the installation process of **Visual Studio Code** for both Windows and Mac operating systems instead. If you prefer to use a different editor, feel free to do so. Major editors like Atom, Sublime, Notepad++, or others will all work fine for this course and is covered in the next lesson.

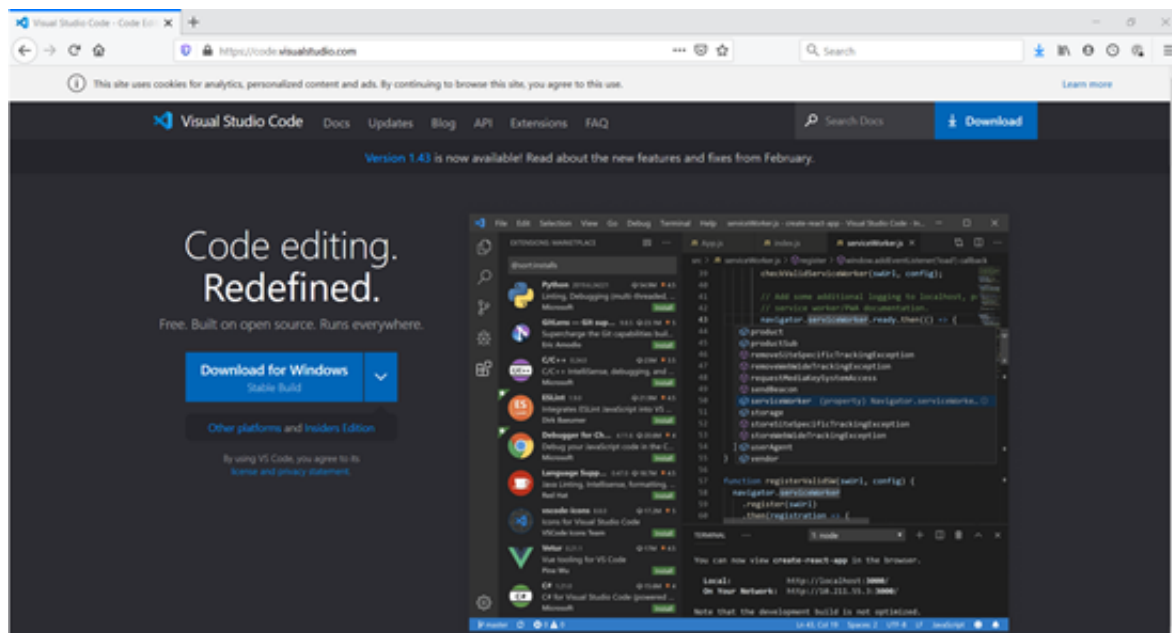
We will start with the Windows installation and finish with the Mac installation so that by the end you will have Visual Studio Code installed on your system, and will be ready to begin coding!

Visual Studio Code

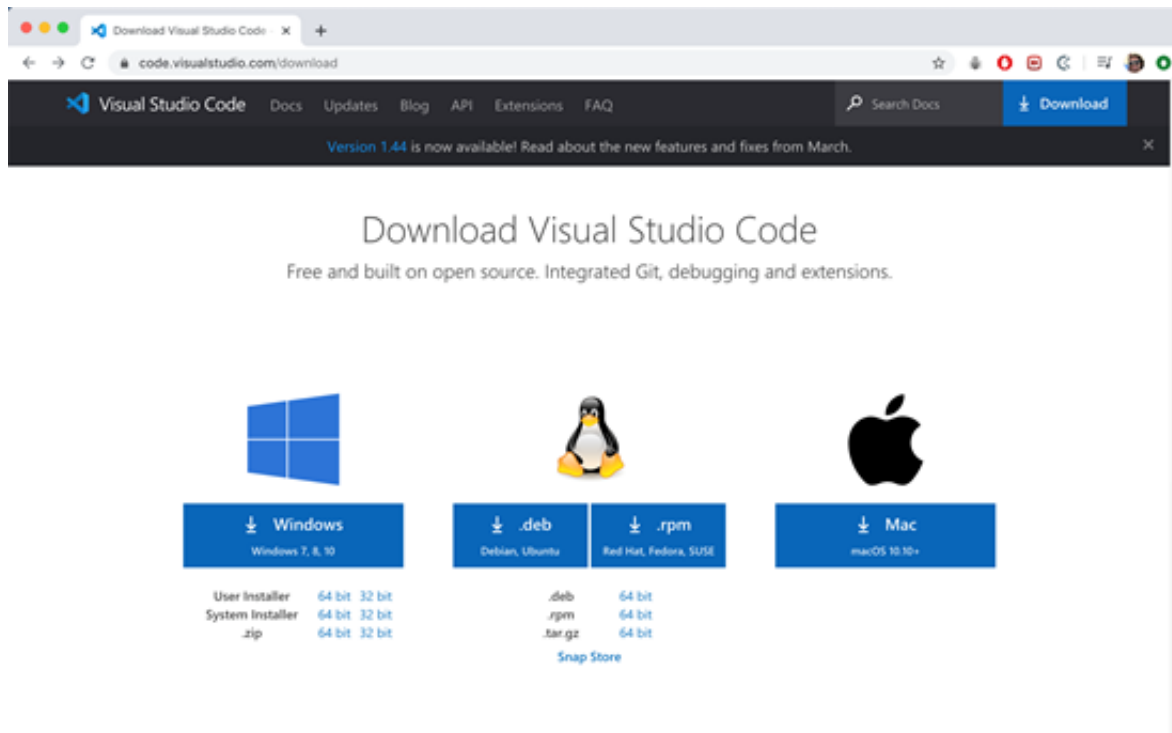
Visual Studio Code, or **VSC** for short, is an IDE, or **Integrated Development Environment**. This is a program that allows users to write, debug, and run code to create programs. VSC is not tailored towards one specific type of program such as Android Studio for Android Apps or Pycharm for Python development. Rather, it acts as a general IDE and can be used for any kind of development. **Through the installation of various plugins and frameworks, you can add support for multiple languages used for various purposes** from web development, to game development, to desktop application development. In summary, VSC is super easy to download and install, supports many languages, and is visually appealing and easy to use.

Installing Visual Studio Code for Windows

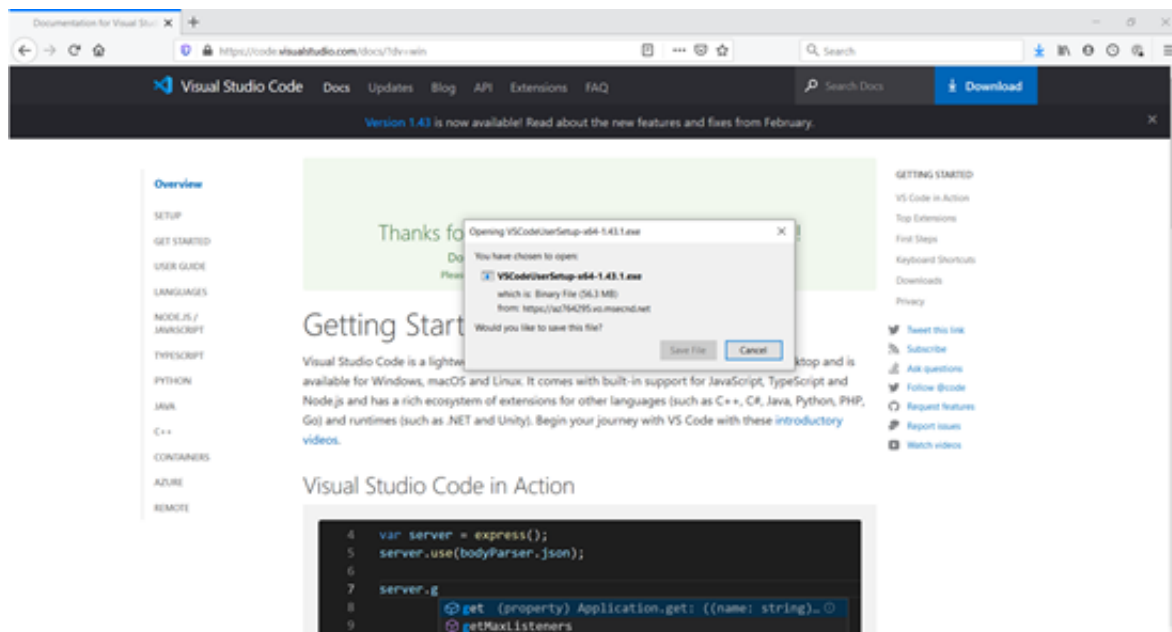
Before we can install VSC, **we must download a version that fits our operating system**. We can start by navigating to the home page found here: <https://code.visualstudio.com/>



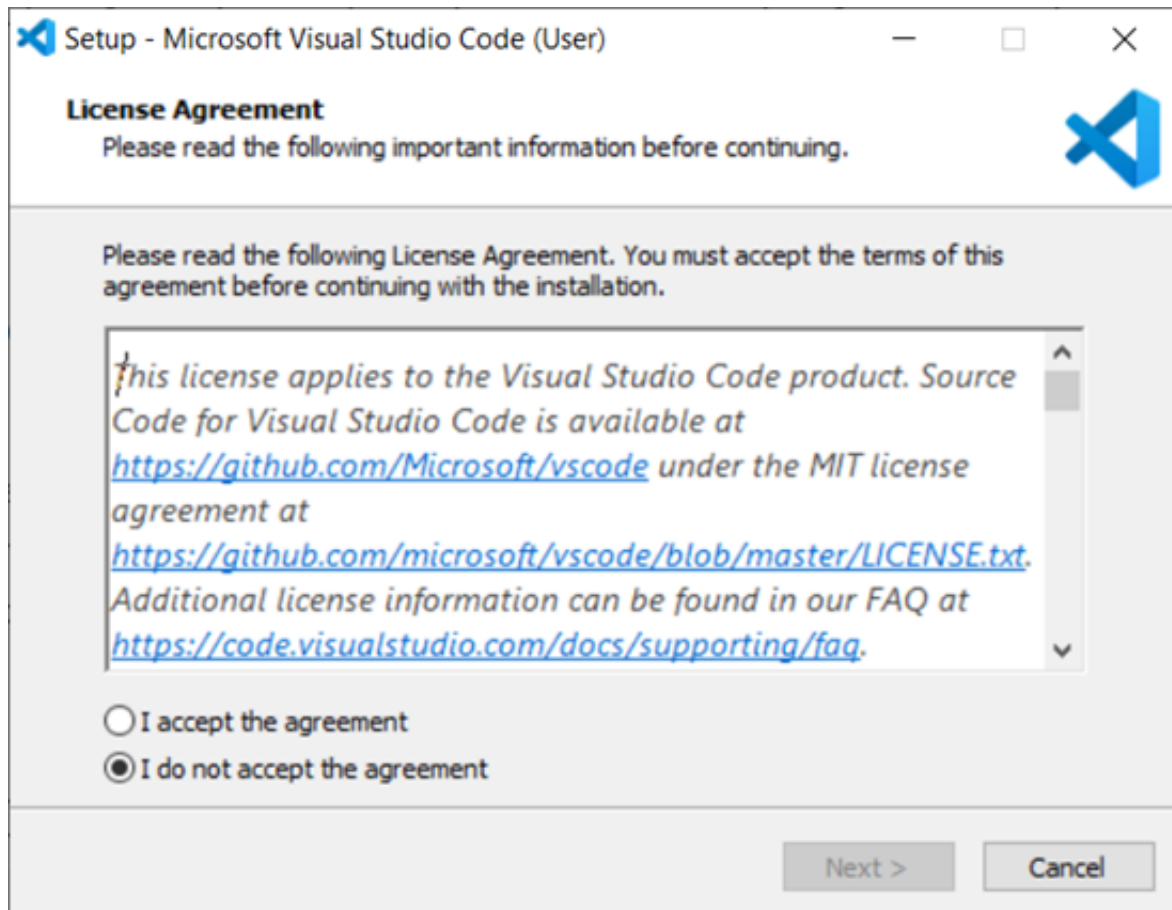
To download the **default/newest version**, click on the **“Download for Windows”** button or navigate to this page to choose a specific download: <https://code.visualstudio.com/download>



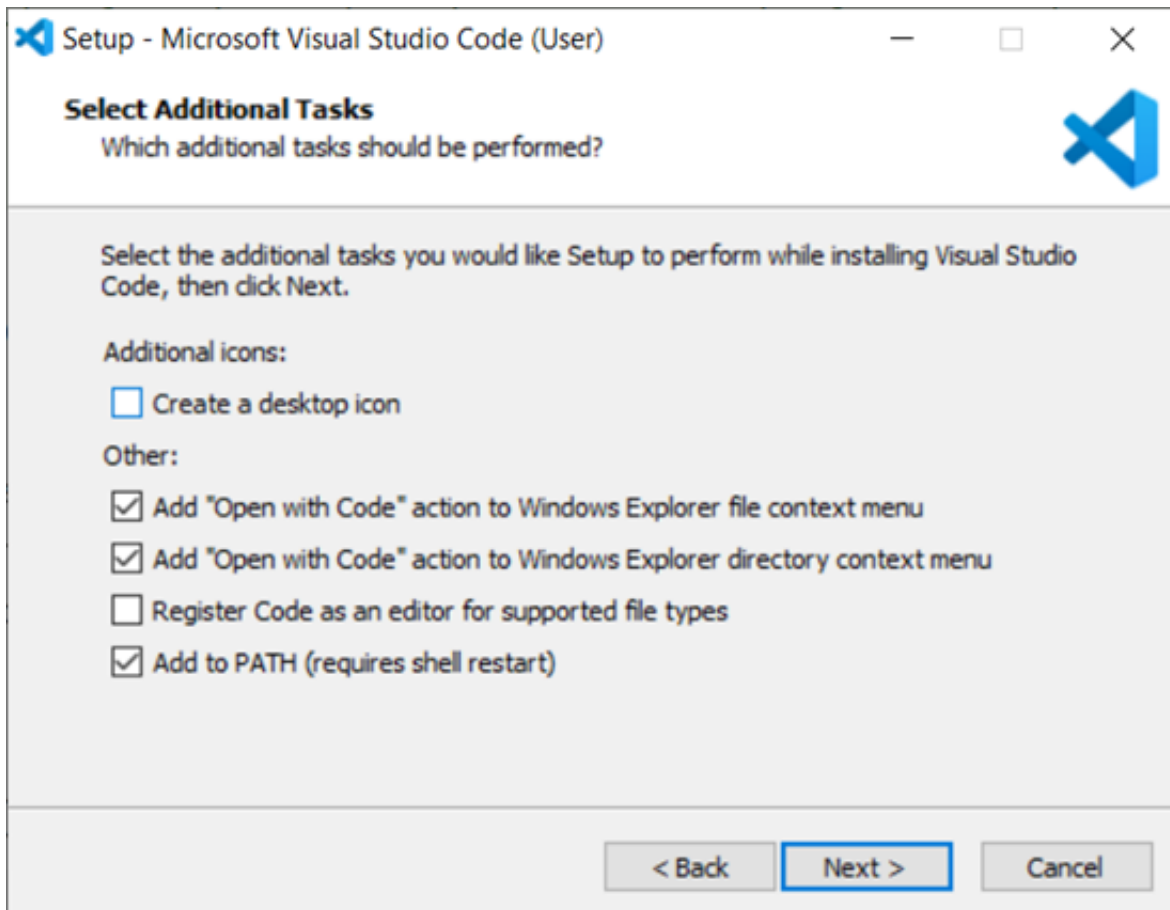
Once you start the download, you should be redirected to the “Getting started” page while the installation wizard downloads. Once done, open the wizard like this:



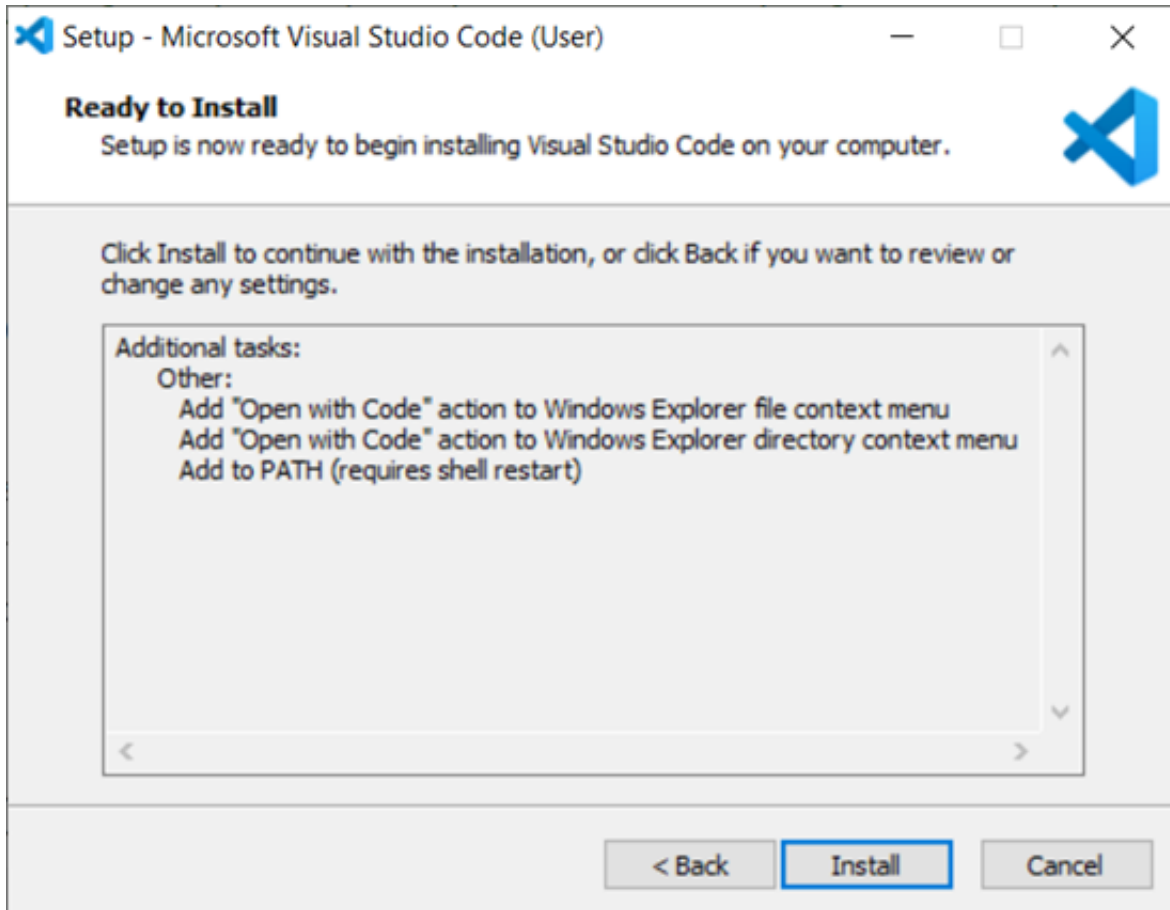
Once the installation wizard starts, **read through and accept the agreement:**



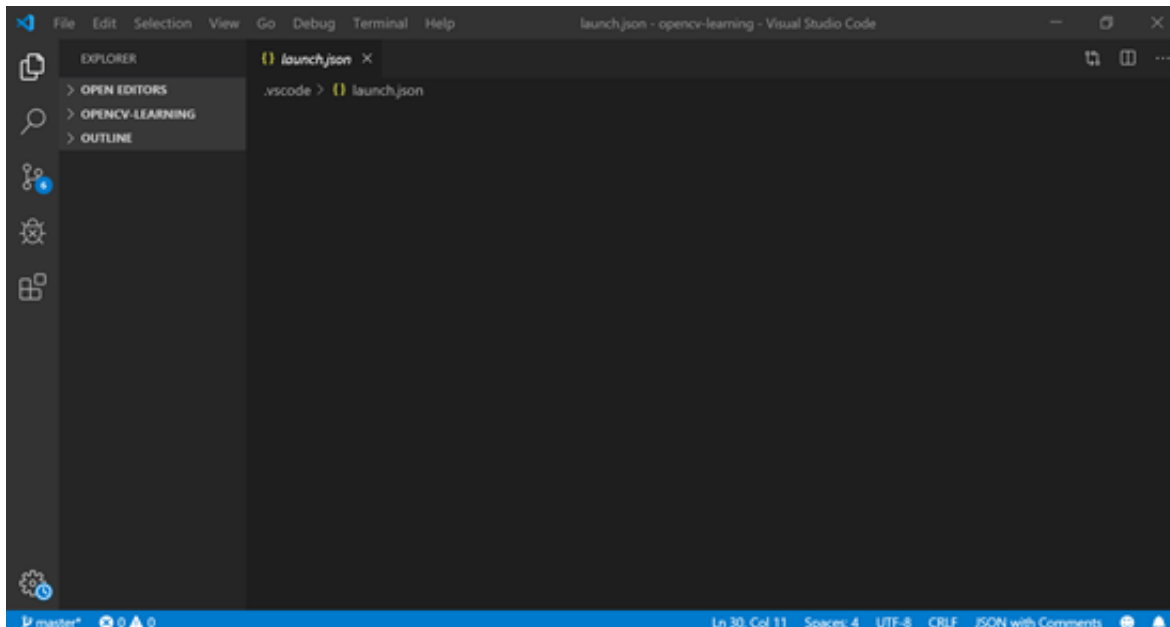
Once you select accept and click next, you will be prompted to select some specific installation properties. Advanced users can change these but **we recommend sticking with these defaults**:



Click next and you will see this prompt:



Once you click Install, you will see a progress bar fill up as the installation proceeds. Once that is done, you can select finish and open up VSC. You should then see an interface that looks like this:

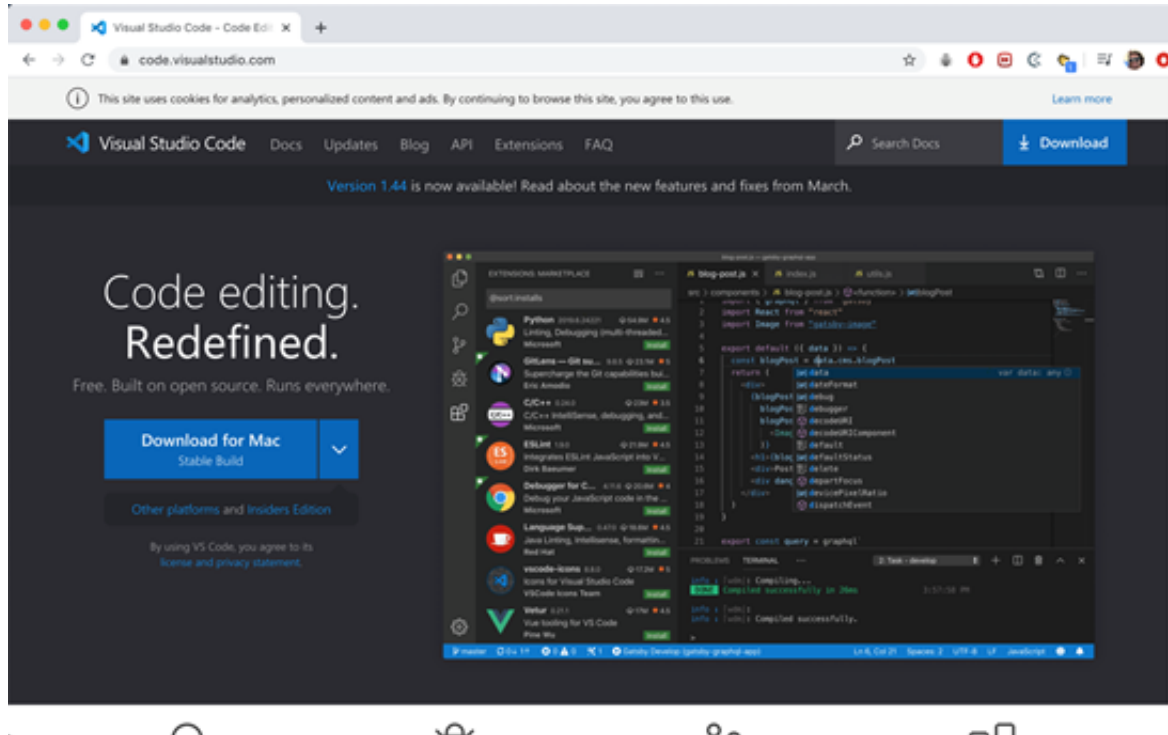


This is VSC in dark mode; you can select the light mode if you want by changing the settings. That's it! You have successfully downloaded and installed VSC and are ready to begin writing and running code!

Installing Visual Studio Code for Mac

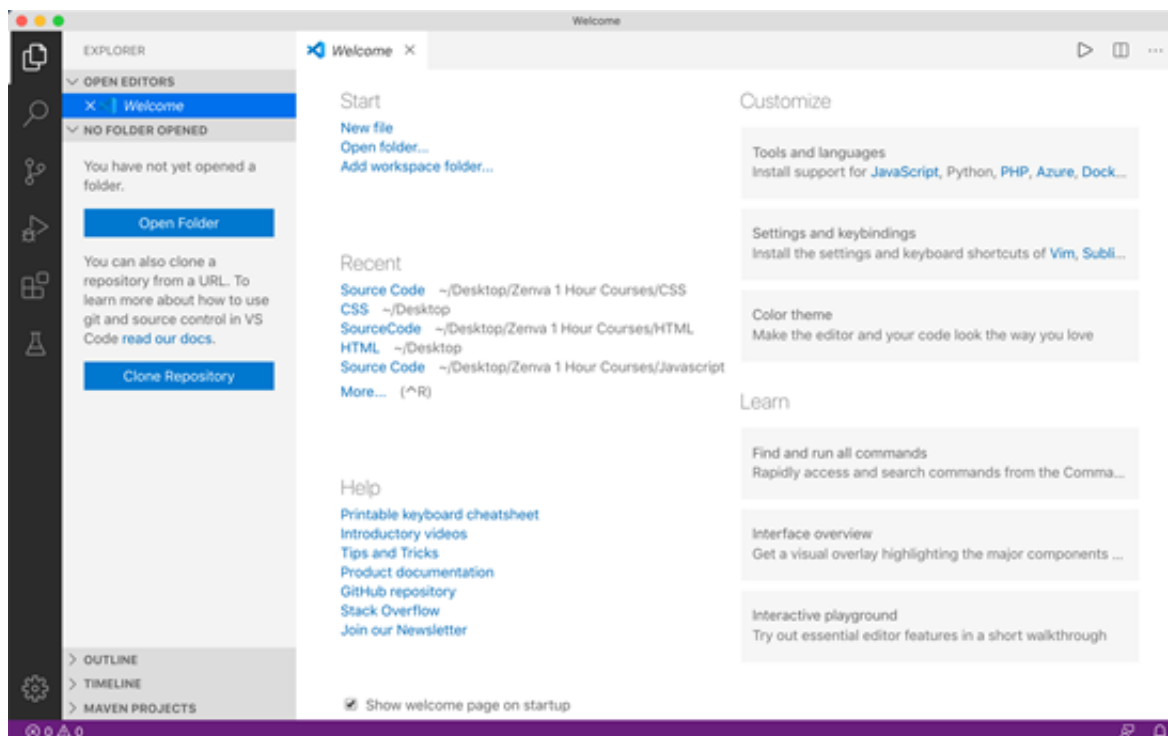
Next up, we will install VSC for Mac.

First, navigate to this website and select **Download for Mac**: <https://code.visualstudio.com/>



Alternatively, you can navigate to this page for a specific build: <https://code.visualstudio.com/download>

This will download a .dmg file and navigate you to the Getting Started page. Once the **.dmg file** finishes downloading, **double click** on it to begin the installation process. There are a few steps to follow here; once the .dmg file has finished opening, it should create a **Visual Studio Code app file** which you can drag and drop into your Applications folder. Once that is done, you can open Visual Studio Code to see something like this:

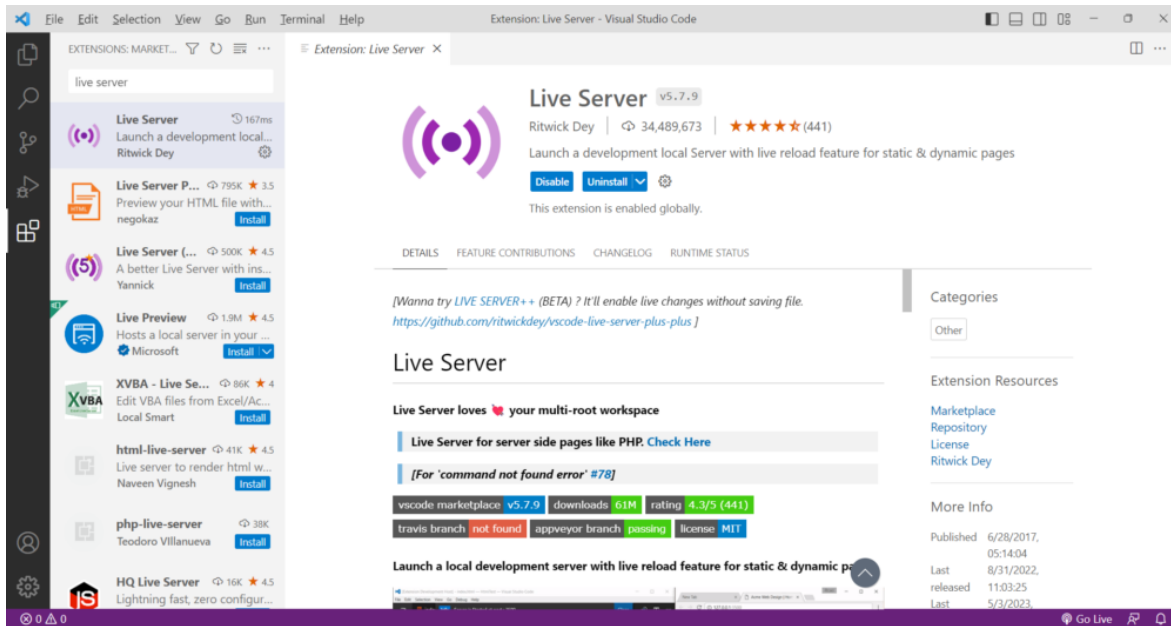


The exact appearance may look slightly different as there are many plugins installed already on the window in this screenshot. That's it! You can now use VSC to write and run code!

Installing Live Server

To launch a local development web server with a live reload feature for static and dynamic pages to playtest our game during development, we can install a VSC extension called **Live Server**.

1. In Visual Studio Code, select **Extensions** in the left-side Activity bar.
2. In the "Search Extensions in Marketplace" field at the top of the Extensions pane, type "live server" and press Enter.
3. Select "Live Server (Ritwick Dey)" in the list.
4. Near the top of the Live Server extension page, click **Install**.



When installation has finished, you'll see a "Go Live" button at the bottom-right of the VSC window – click this button whenever you're ready to playtest your Phaser project code in the web browser. Simple.

Summary

You now have access to the VSC software and can use this to write code. You can search for any specific extensions and plugins by clicking on the "Extensions" button and can create or open files for editing under the "File" dropdown menu.

Happy coding!

Notice

If you decide to use **Visual Studio Code (VSC)** and **Live Server**, as set up in the previous lesson, you may skip this lesson. Otherwise, this lesson provides alternatives for the code editor and web server.

Learning Goals

- Basic Phaser development environment.
- Setting up a web server.

In order for you to start making games with Phaser you will need to have a **web browser** (Google Chrome will be used for this course), **code editor**, and a **web server**.

There are many different code editors out there that are free to use, and it doesn't matter which one you choose.

- A list of code editors that are free
 - Brackets(<https://brackets.io>)
 - Atom(<https://atom.io>)
 - Sublime text(<https://www.sublimetext.com>)
 - Visual Studio Code(<https://code.visualstudio.com/>)

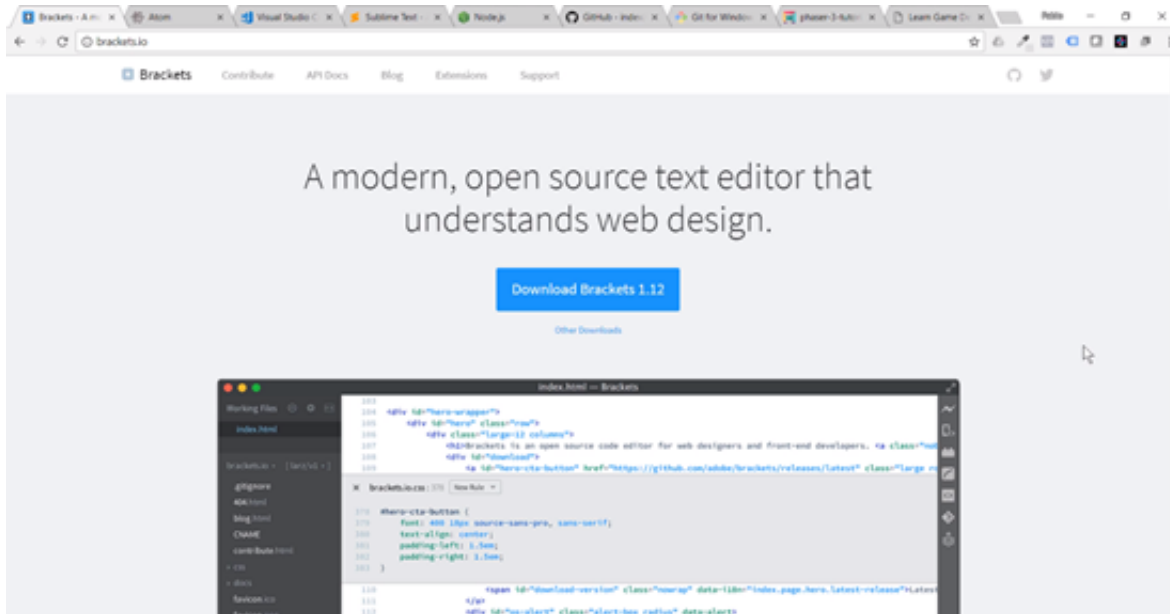
Code Editors

- Brackets (<https://brackets.io>)
- Atom (<https://atom.io>)
- Sublime Text (<https://www.sublimetext.com/>)
- Visual Studio Code (<https://code.visualstudio.com/>)

A web browser is a program that will receive requests from the browser, and we will send, it will serve files as a response.

A browser doesn't allow you to load files from the file protocol due to security reasons. The web would be a very dangerous place if random websites could get access to the files on your computer. This is why you cannot just go and open a Phaser game by double clicking on the index.html file.

You have to load the game through a **web server**. The simplest way to get a web server up and running is to install the Brackets code editor.



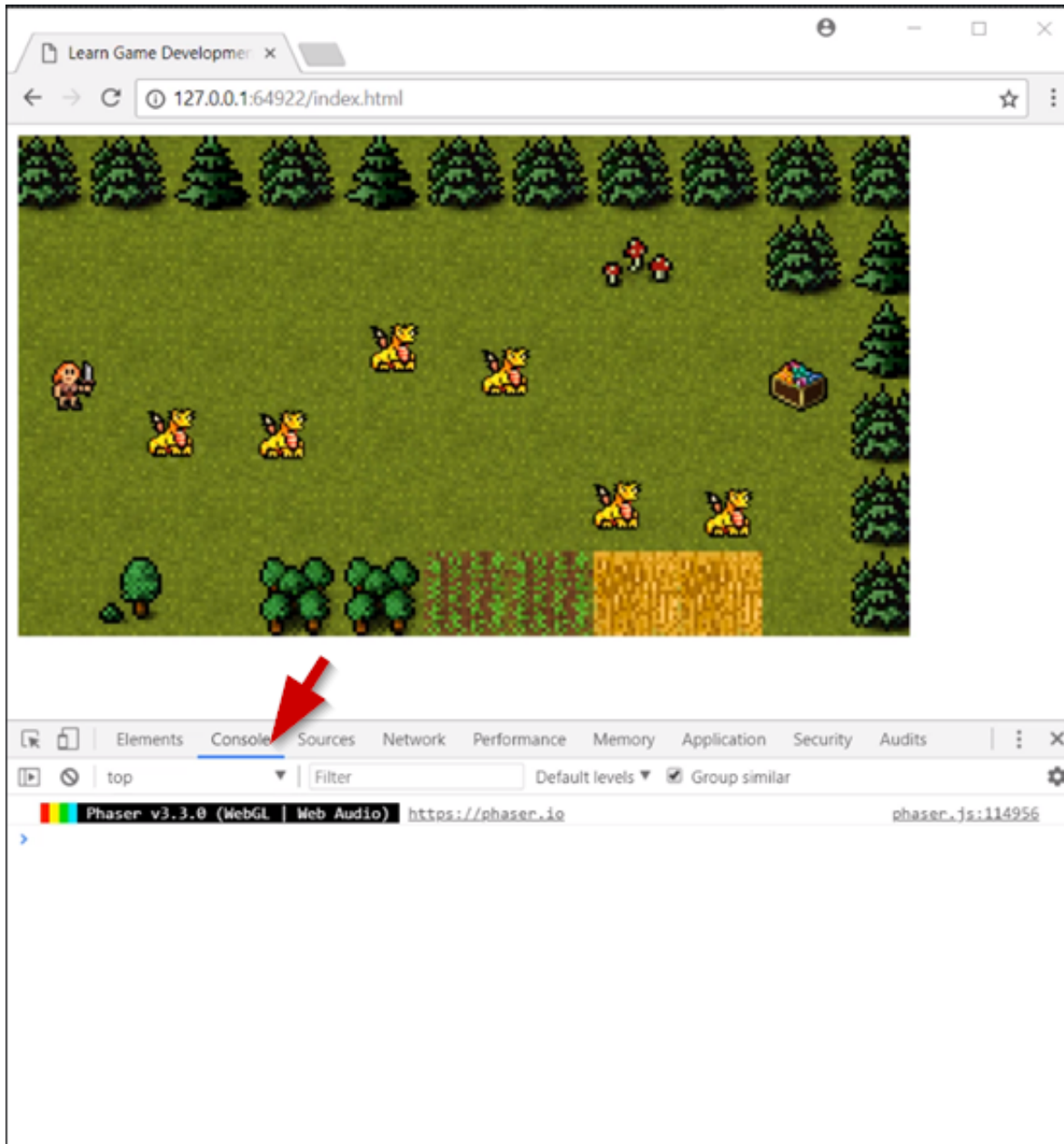
You can **download** Brackets from here: <http://brackets.io>

The **direct link** to download Brackets is here: [Download Brackets](#)

Brackets comes with a built-in web server and it is free to use.

- Web Server-Brackets
 - Download and install Brackets (<https://brackets.io>)
 - File-Open Folder
 - Live preview

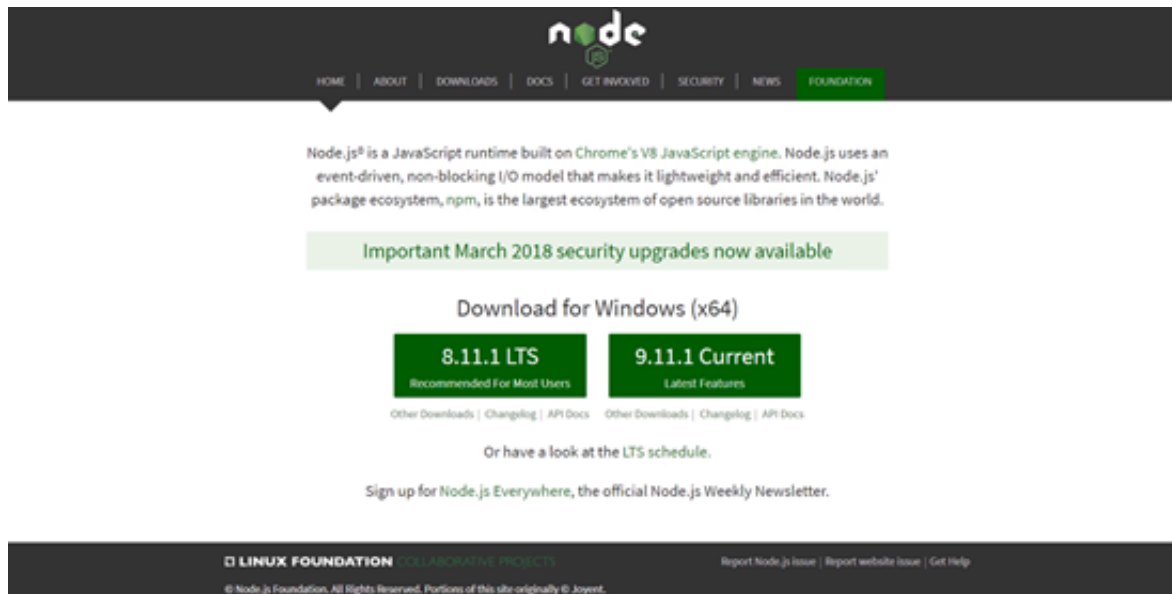
Always keep the Chrome Developer Tools open when working.



There is a more advanced way of using a web server and this involves using a called http server.

The first step is to install **Node JS**, and you can download Node JS from here: <https://nodejs.org/en/>

The direct link to **download** Node JS is here: [Node JS Download](#)



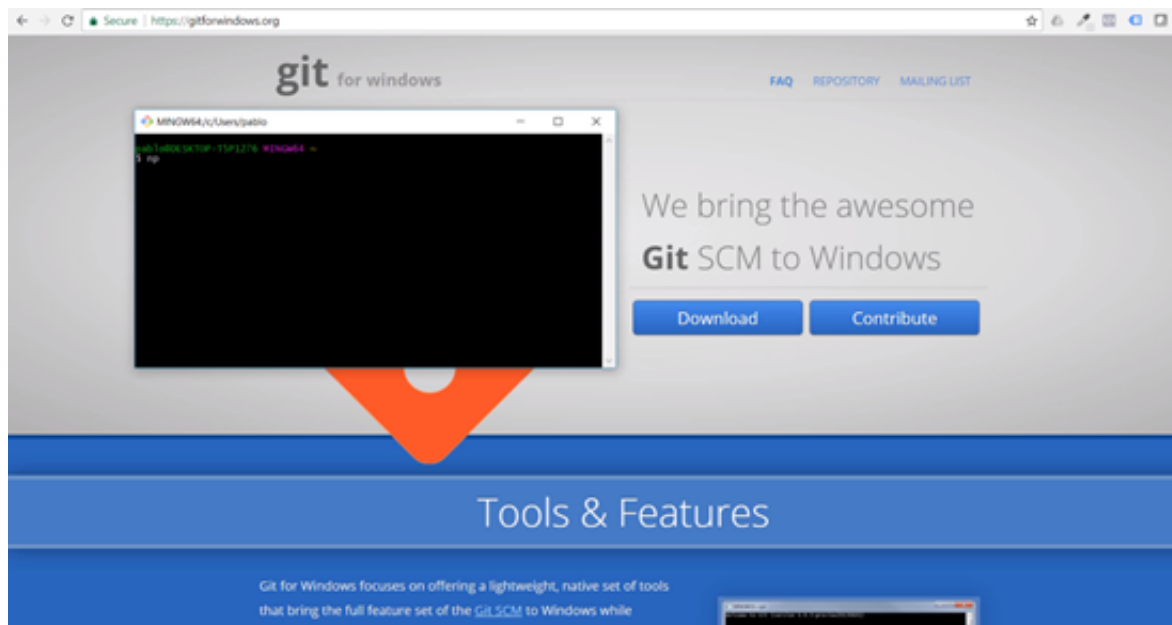
Node JS is an application that allows you to run JavaScript code on a server, in this case your computer.

For Window Users Only: Once you have downloaded Node JS you will need to download Git Bash, and this will give you access to a terminal on your system.

Mac users will already have access to the Mac terminal.

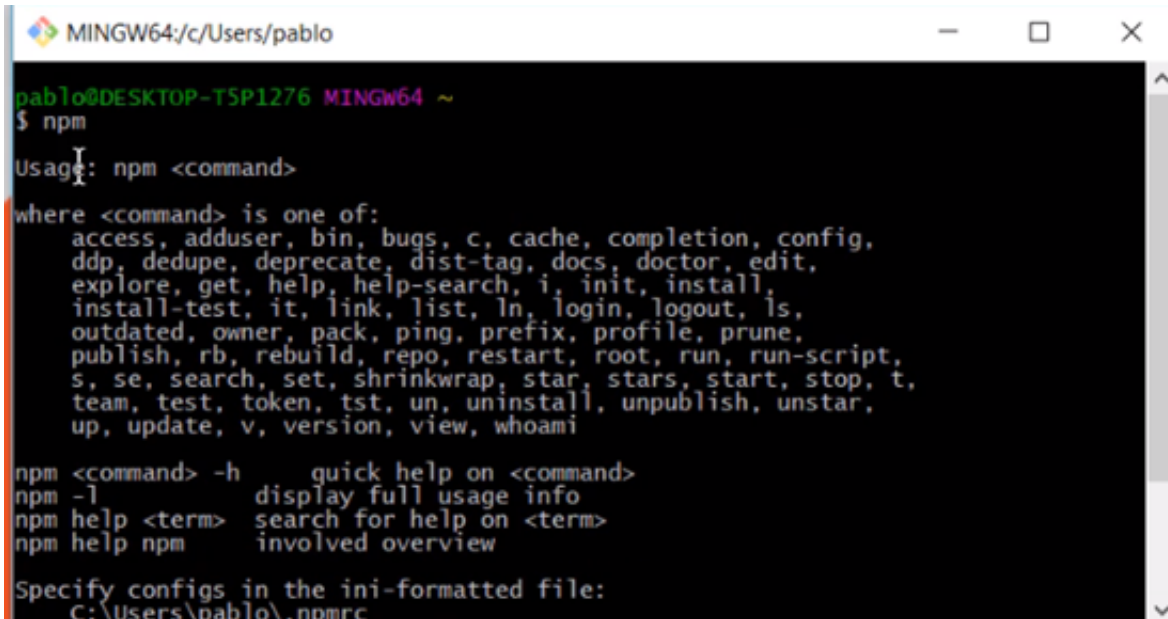
You can download **Git Bash** for windows here: <https://gitforwindows.org/>

The direct link to **download** Git Bash for windows is here: [Git Bash](#)



The first step is type **“npm”** in the terminal

If you type this and it is installed you will see this:



```
MINGW64/c/Users/pablo
pablo@DESKTOP-T5P1276 MINGW64 ~
$ npm
Usage: npm <command>

where <command> is one of:
  access, adduser, bin, bugs, c, cache, completion, config,
  ddp, dedupe, deprecate, dist-tag, docs, doctor, edit,
  explore, get, help, help-search, i, init, install,
  install-test, it, link, list, ln, login, logout, ls,
  outdated, owner, pack, ping, prefix, profile, prune,
  publish, rb, rebuild, repo, restart, root, run, run-script,
  s, se, search, set, shrinkwrap, star, stars, start, stop, t,
  team, test, token, tst, un, uninstall, unpublish, unstar,
  up, update, v, version, view, whoami

npm <command> -h      quick help on <command>
npm -l               display full usage info
npm help <term>      search for help on <term>
npm help npm         involved overview

Specify configs in the ini-formatted file:
  C:\Users\pablo\.npmrc
```

Once you verify that the npm is installed you can then type **"npm install http-server -g"** this is the name of the package that we want installed for us.

We now need to navigate and find our folder where the game is located.

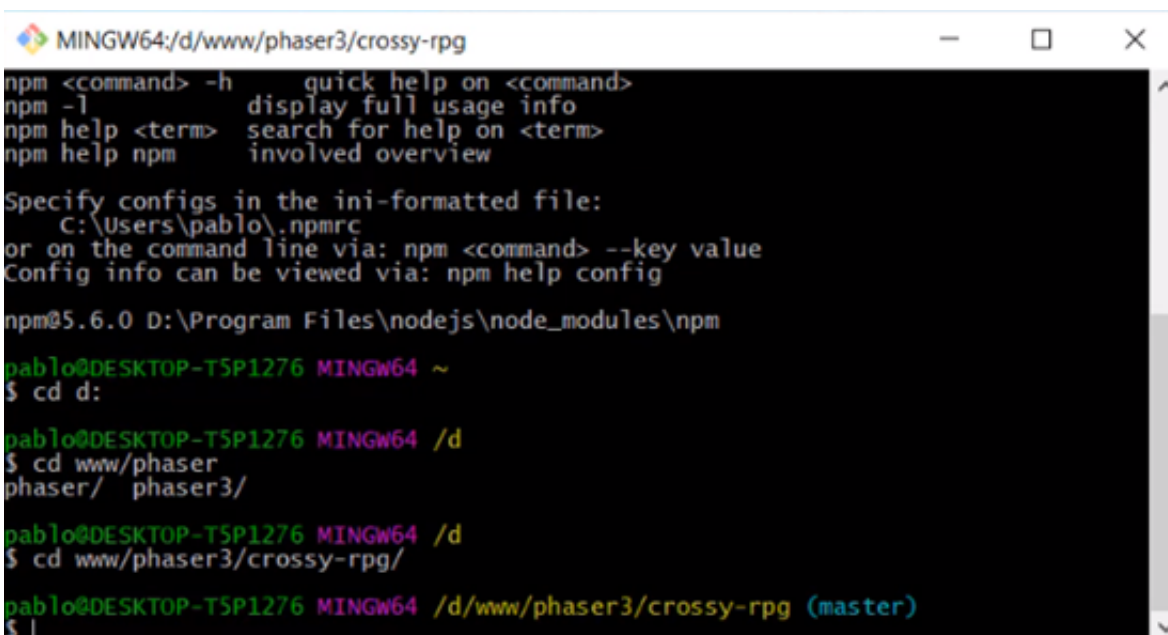
In my case the game is located in the D drive, the **www folder>Phaser3>crossy-rpg** this will be different in your case.

If you have issues with using the command line options here you can just use Brackets, Brackets is much easier to use and doesn't require this extra setup for the http server setup.

So in the terminal you type **"cd d:"** this brings you to the D drive.

Then type **"cd www/phaser3/crossy-rpg/"**

Now you should be inside the correct folder in the terminal.



```
MINGW64/d/www/phaser3/crossy-rpg
npm <command> -h      quick help on <command>
npm -l               display full usage info
npm help <term>      search for help on <term>
npm help npm         involved overview

Specify configs in the ini-formatted file:
  C:\Users\pablo\.npmrc
or on the command line via: npm <command> --key value
Config info can be viewed via: npm help config

npm@5.6.0 D:\Program Files\nodejs\node_modules\npm
pablo@DESKTOP-T5P1276 MINGW64 ~
$ cd d:

pablo@DESKTOP-T5P1276 MINGW64 /d
$ cd www/phaser
phaser/ phaser3/

pablo@DESKTOP-T5P1276 MINGW64 /d
$ cd www/phaser3/crossy-rpg/

pablo@DESKTOP-T5P1276 MINGW64 /d/www/phaser3/crossy-rpg (master)
$ |
```

Now that you are inside the correct folder you then type **"http-server"** and that should launch the web server for that particular folder.

You should now see that the server is available in different URLs.

```
or on the command line via: npm <command> --key value
Config info can be viewed via: npm help config

npm@5.6.0 D:\Program Files\nodejs\node_modules\npm

pablo@DESKTOP-T5P1276 MINGW64 ~
$ cd d:

pablo@DESKTOP-T5P1276 MINGW64 /d
$ cd www/phaser
phaser/ phaser3/

pablo@DESKTOP-T5P1276 MINGW64 /d
$ cd www/phaser3/crossy-rpg/

pablo@DESKTOP-T5P1276 MINGW64 /d/www/phaser3/crossy-rpg (master)
$ http-server
Starting up http-server, serving ./
Available on:
  http://192.168.99.1:8080
  http://192.168.1.103:8080
  http://127.0.0.1:8080
Hit CTRL-C to stop the server
```

You then can copy one of the URLs and go back into Google Chrome or whatever web browser you are using and paste the copied code into the navigation bar.

```
or on the command line via: npm <command> --key value
Config info can be viewed via: npm help config

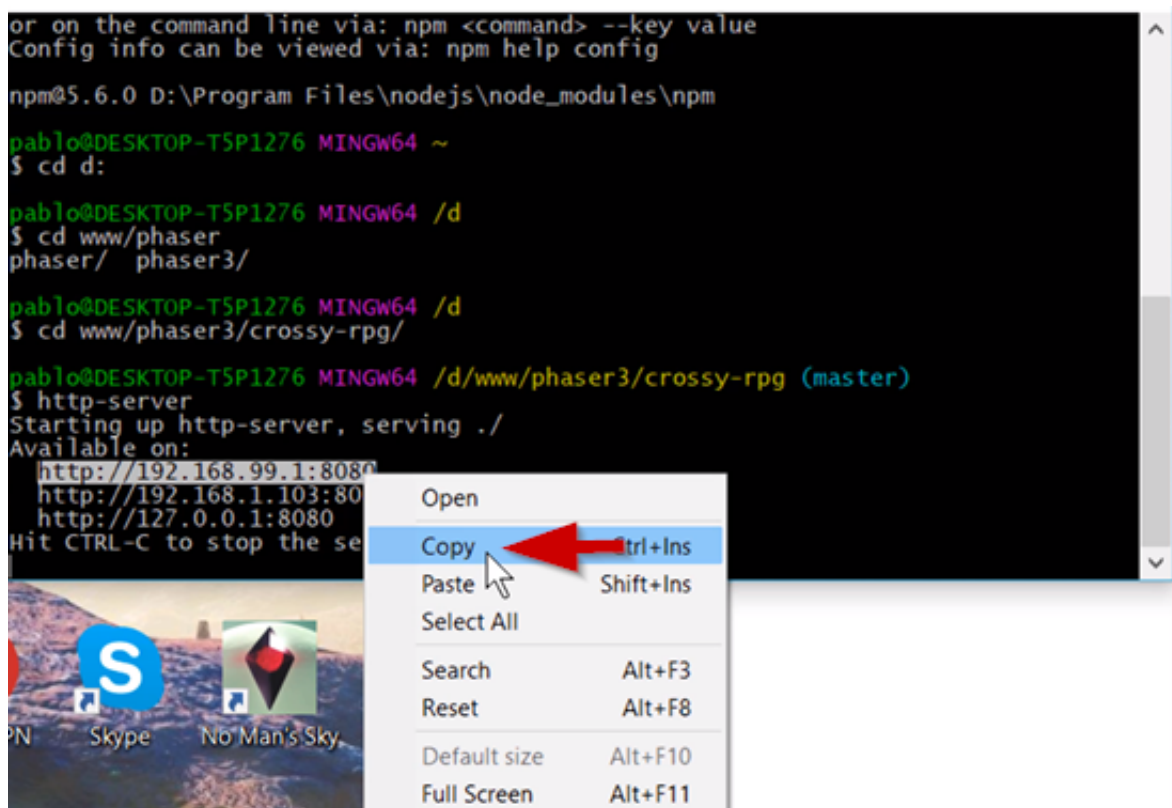
npm@5.6.0 D:\Program Files\nodejs\node_modules\npm

pablo@DESKTOP-T5P1276 MINGW64 ~
$ cd d:

pablo@DESKTOP-T5P1276 MINGW64 /d
$ cd www/phaser
phaser/ phaser3/

pablo@DESKTOP-T5P1276 MINGW64 /d
$ cd www/phaser3/crossy-rpg/

pablo@DESKTOP-T5P1276 MINGW64 /d/www/phaser3/crossy-rpg (master)
$ http-server
Starting up http-server, serving ./
Available on:
  http://192.168.99.1:8080
  http://192.168.1.103:80
  http://127.0.0.1:8080
Hit CTRL-C to stop the se
```

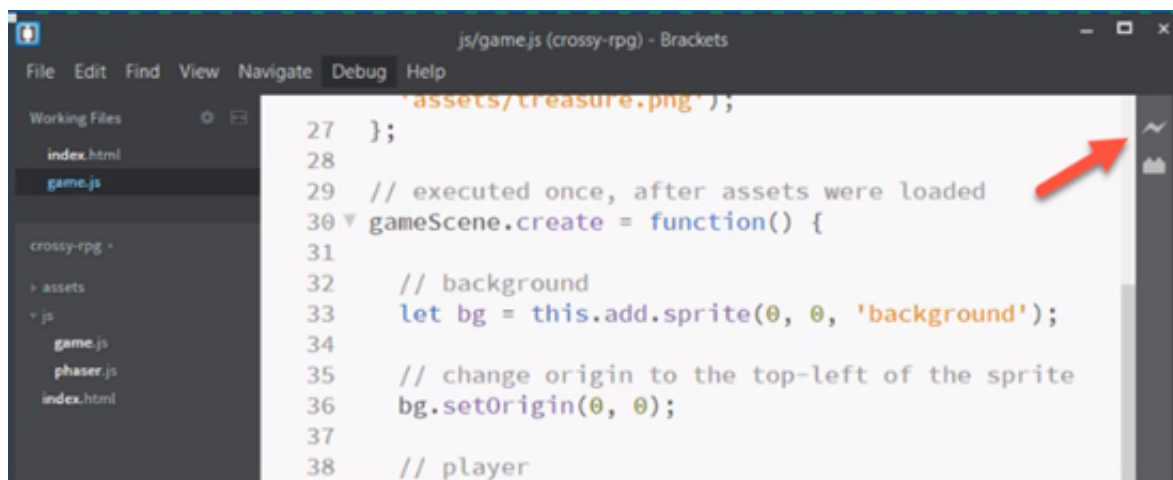




And now the game will be working just like it should.

Learning Summary

- There are three basic requirements for Phaser development
 - Web browser
 - Code editor
 - Web Server
- A list of code editors that are free
 - Brackets(<https://brackets.io>)
 - Atom(<https://atom.io>)
 - Sublime text(<https://www.sublimetext.com/>)
 - Visual Studio Code(<https://code.visualstudio.com/>)
- Web Server-Brackets
 - Download and install Brackets (<https://brackets.io>)
 - File-Open Folder
 - Live preview



- Web Server-http-server
 - Download and install Node.js (<https://nodejs.org>)
 - (Windows Only) install Git Bash (<https://gitforwindows.org/>)



- Open terminal, install http-server: `npm install http-server -g`
- Navigate to the project folder, run the server: `http-server`

Learning Goals

- Game Design Documents
- Importance of scoping
- Outlining game requirements

Game design documents(GDD's) are used in the game industry to **describe and communicate the concepts and requirements of a game** that will be developed.

These documents can be very short or very long. The length will depend on the type of game and scope of the game as well.

The GDD's allow you to see everything in one place for the game that is being developed. It makes it easy for people who are developing a game on a team to get an idea of what the game should be, or other people who might just be interested in the development of the game. They can look at the GDD and see exactly what mechanics are going to be in the game and even see the type of art being used in the game.

GDD's can be used if you are working on a team or if you are just a solo developer.

You will want to include the **concept** of the game, the game **mechanics**, the **theme**, the **genre**, the **targeted platforms**, and any **art or sound assets** in the GDD.

A simple template for a GDD:

GAME DESIGN DOCUMENT

Game concept

Target platforms

UI / Player controls

Game mechanics

You can use just about word processing software to write the GDD in.

A GDD is considered to be a **“live”** document because it should be constantly updated and everyone working on the game should have access too the document at all times.

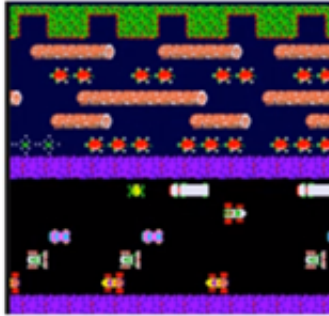
Learning Summary

- Game Design Documents (GDD)
 - Short documents to describe a game concept and requirements.
 - Used by and number of people (1 or many)
 - Includes:
 - Concept
 - Mechanics/UI
 - Platforms
 - Assets

GAME DESIGN DOCUMENT

Game concept

It's casual, 2D, "Frogger" style, where the player controls a Valkyrie who needs to reach a treasure and avoid dragons.



Target platforms

- Cross-platform: desktop and mobile
- Browser game

UI / Player controls

- Mouse click / touch screen
- When the player clicks/touches the screen, the character moves forward

Game mechanics

- Dragons moving up and down (bouncing movement)
- If the is hit by a dragon, the game restart
- The player wins if they reach the treasure. The game will restart
- Player moves in one direction|

Assets

- Style: pixel art, 8-bit style, retro RPG
 - Sprites:
 - Background
 - Player
 - Enemy
 - Treasure
- Challenge
 - Make a GDD of your game idea
 - Share it in the comments

The instructions in the video for this particular lesson have been updated, please see the lesson notes below for the corrected code.

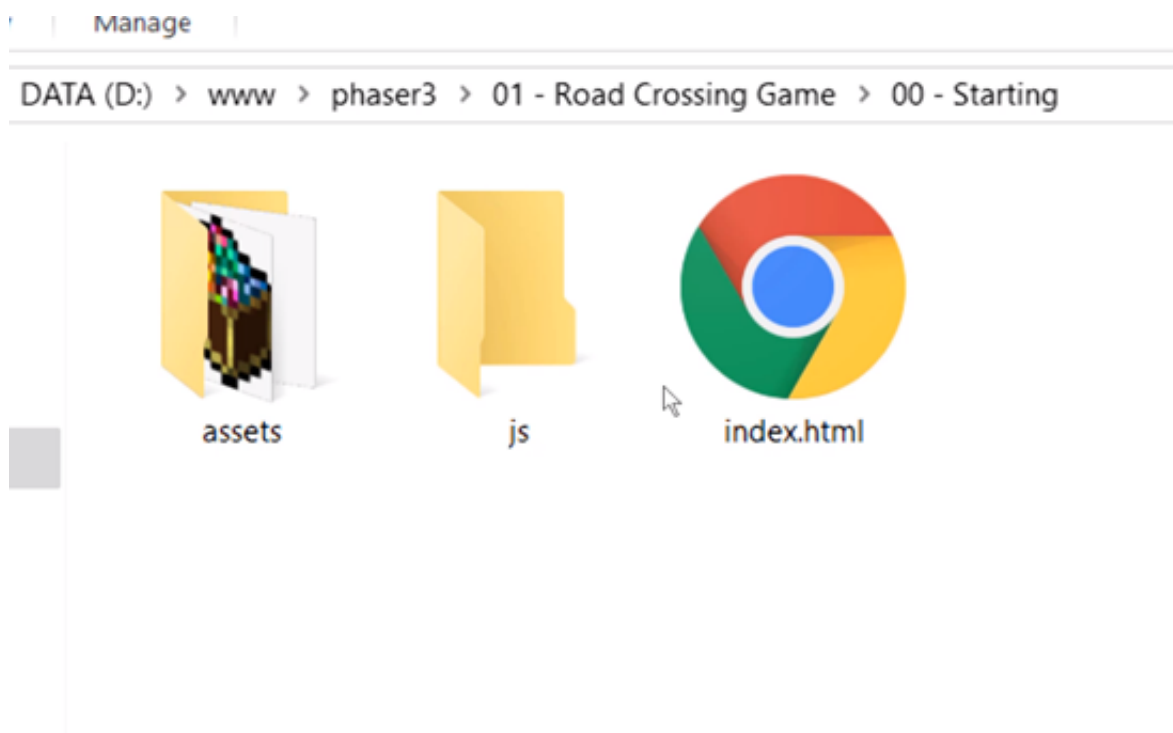
Learning Goals

- Including Phaser in a project.
- Passing configuration to a new game.
- Creating a Phaser game with a scene.

In this lesson we will begin creating the new Phaser game.

It will be an empty game on an empty scene, and you will learn what a scene is.

Download the files for the course, and make sure you find the folder named **“oo-Starting”**



You will be building the game along as you watch the lessons in the course.

The **js** folder is empty and this is where all the **JavaScript files** will go.

There is also a folder called **assets** and this contains all the images for the project.

You can **open the index.html file in your code editor**. I am using Atom in this course for my code editor.

This is a simple file that doesn't really have anything.

We will need to **start by including the Phaser library**.

You can find the Phaser library at phaser.io.

The direct link is here: [Phaser Download](https://github.com/phaserjs/phaser)



If you want to put your game on the internet you will always go with the **minified version**. This version has all the comments stripped out of it.



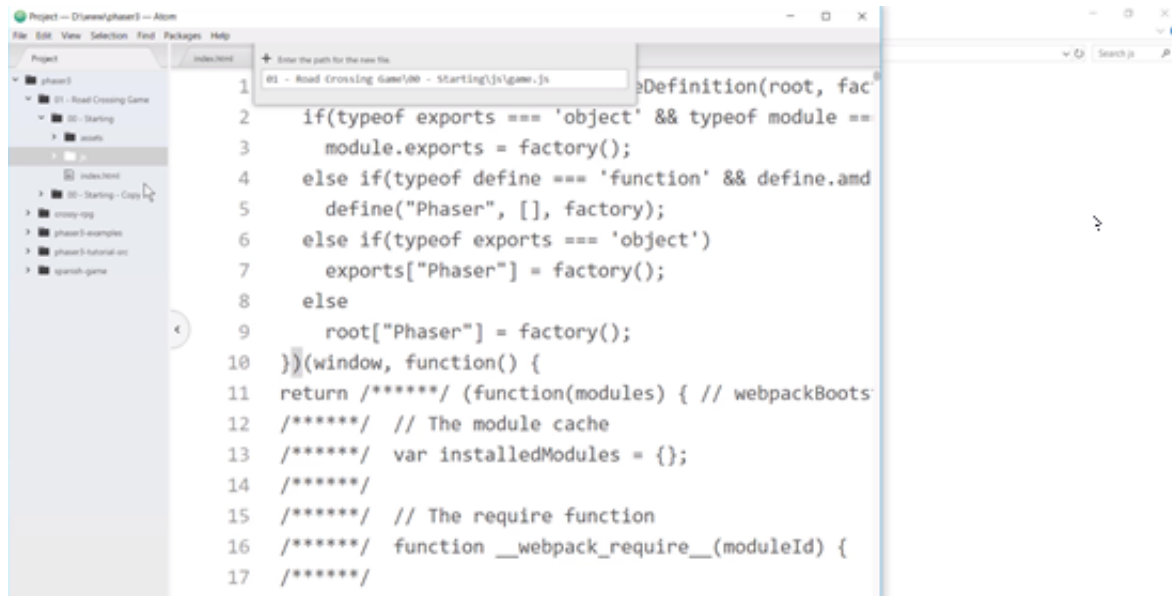
You need to **save the min.js file** in the JavaScript folder.

Then you need include the file in the index.html file.

See the code below and follow along:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Learn Game Development at Zenva.com</title>
</head>
<body>
  <script src="js/phaser.js"></script>
</body>
</html>
```

Create a new file called **"game.js"** and this is where the game code will go.



Make sure to **include that file in the index.html file**, but after the Phaser file. See the code below and follow along:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Learn Game Development at Zenva.com</title>
</head>
<body>
  <script src="js/phaser.js"></script>
  <script src="js/game.js"></script>
</body>
</html>
```

Inside the **game.js file** there are three things that need to happen. We need to **create a new scene**, set the **configuration of the game**, and **create the new game** and pass the **configuration** to it.

A **scene** is where all the action takes place. This is where all the game characters, enemies, and everything in the game will go in the scene. In Phaser you can actually have multiple scenes. You can use scenes to represent different screens, or in some cases different levels.

See the code below and follow along:

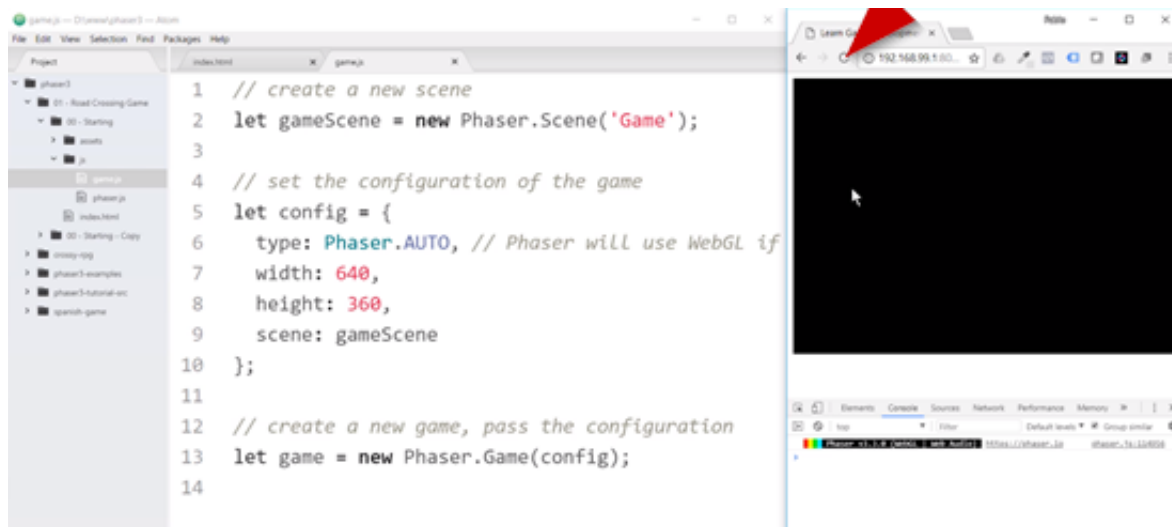
```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  width: 640,
```



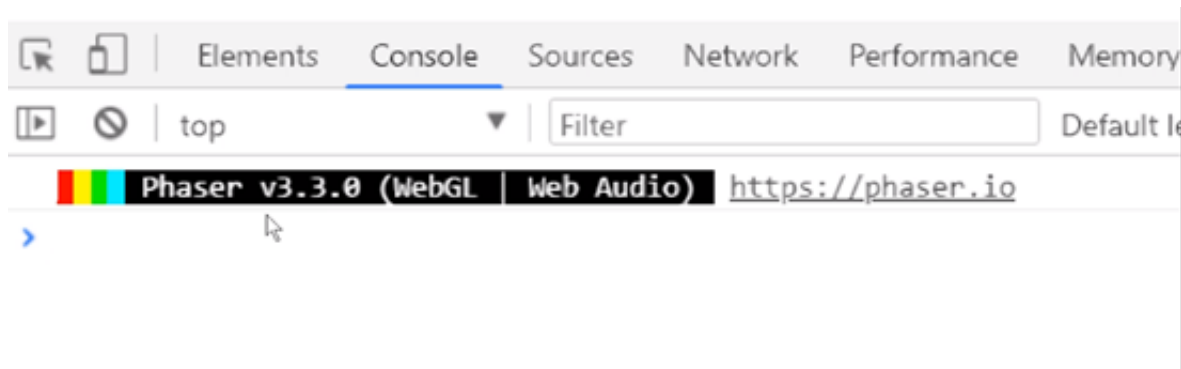
```
height: 360,  
scene: gameScene  
};  
  
// create a new game, pass the configuration  
let game = new Phaser.Game(config);
```

Then test out the changes by **refreshing** the web page:



There isn't much happening currently, there is a black screen of the size that we defined.

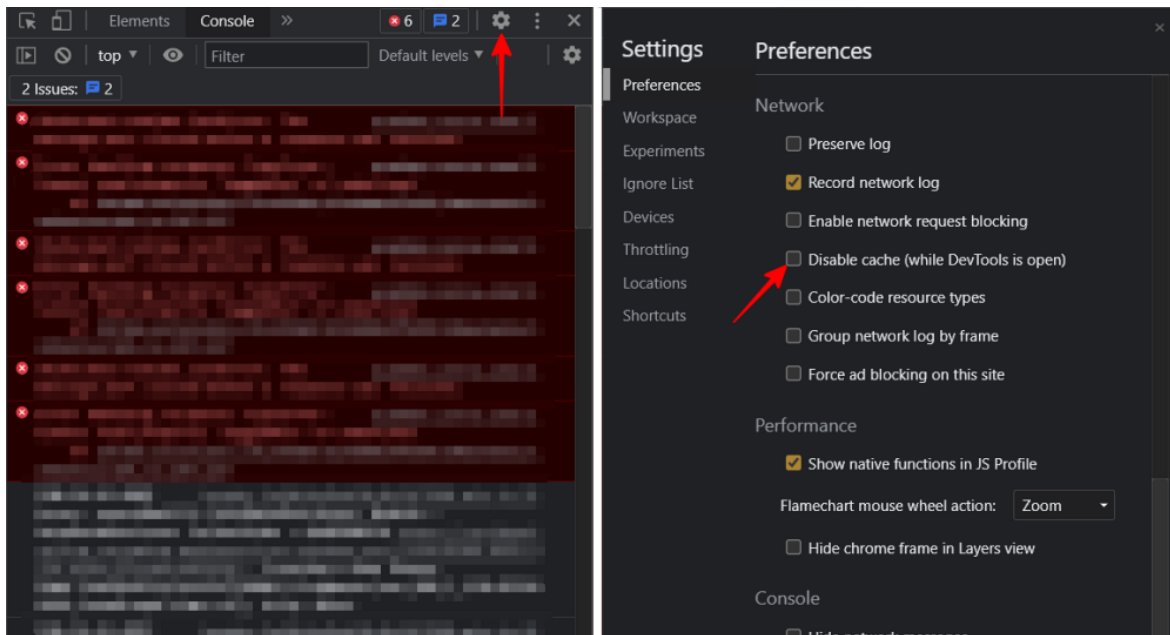
You will see that there is an indication in the **Console window** that Phaser is indeed working though.



The following instructions have been updated, and differs from the video:

In order to see changes to your code in the web browser, you must either reload the page and ignore the cached content or you need to disable browser caching (while the dev tools are open). Here is the shortcut for Chrome to reload the current page, ignoring cached content: Shift + F5 or Ctrl + Shift + r

To disable browser caching, open the developer tools (Ctrl + Shift + i or F12 in Chrome) and follow the indications below. You can refer to **Lesson 13** at the 6:51 mark for further explanation as well.



In the next lesson, we will be adding images to the screen.

Learning Summary

- Obtaining Phaser
 - Phaser homepage(<http://phaser.io>)
 - Version: 3.x
 - Options:
 - Direct Download
 - CDN
 - Github
 - npm
- Creating a new game
 - Create a scene
 - Configuration
 - Create game object



```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available
  width: 640,
  height: 360,
  scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Corrections

At 11'00", I say "340" when I should have said "640" (this was coded correctly on screen).

At 15'14" on the Slide "Coordinate System", the text should instead read:

X coordinate: positive to the LEFT, negative to the RIGHT

Y coordinate: positive DOWNWARDS, negative UPWARDS

Learning Goals

- Phaser's preload and create methods.
- Rendering a sprite.
- Coordinate system.
- Sprite origin.

In this lesson you will learn how to display images on the screen using Phaser. The images that will be used during this lesson can be downloaded from the course. They are in the assets folder. There are a total of 4 images in the folder: background, dragon, player, and treasure.

We will begin by loading the background, and the loading will occur inside of our scene.

The diagram for the **life cycle of a scene**:



The **init() method**- called once this is used to initiate certain parameters of your scene. This method is not always used.

preload() method-all asset preloading takes place. This means that Phaser will load all of the images, all the audio files, and other external files you may want to use in your game. All of the files will be loaded to memory so that they can be used without any delay. When you want to show an enemy or character, you don't want the player to have to wait until that image is loaded.

create() method-called once and this is actually where you create the sprites and display them on the screen.

update() method-we will look at this method later on in the course, but it is called on each frame during game play. This method is used when things need to be checked all the time.

We will now create the preload method for the scene. See the code below and follow along:

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
};
```



```
// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  //bg.setOrigin(0,0);

  // place sprite in the center
  bg.setPosition(640/2, 360/2);

  let gameW = this.sys.game.config.width;
  let gameH = this.sys.game.config.height;

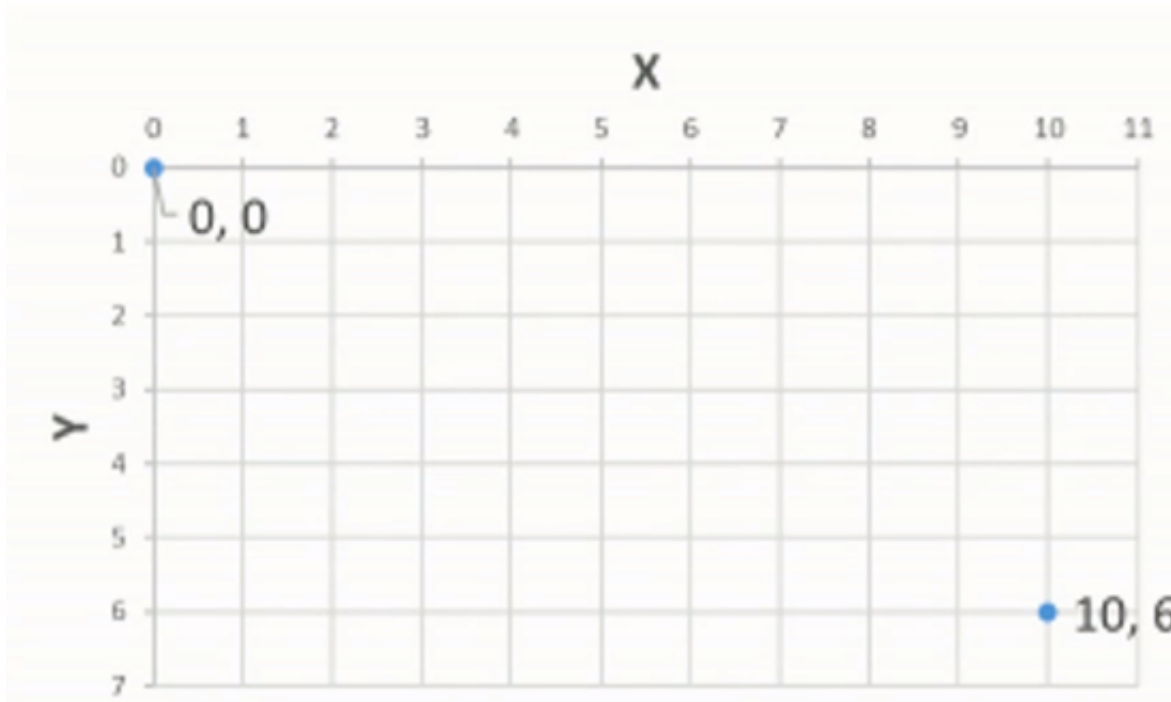
  console.log(gameW, gameH);

  console.log(bg);
  console.log(this);
};

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  width: 640,
  height: 360,
  scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

In Phaser games a **two dimensional coordinate system** is used. There is an X axis that goes from left to right. There is a Y axis that goes from top to bottom. The **origin(0,0)** is always at the top left corner of the game.



If we said we wanted the player to begin in position (5,4) What we are saying is that the center of our sprite will go in that position.

- **X:** Positive to the right, negative to the left.
- **Y:** Positive upwards, negative downwards.

By default the origin is the middle point on X and the middle point on Y.

You can use the **bg.setOrigin(0,0)** method to change the origin of sprites.

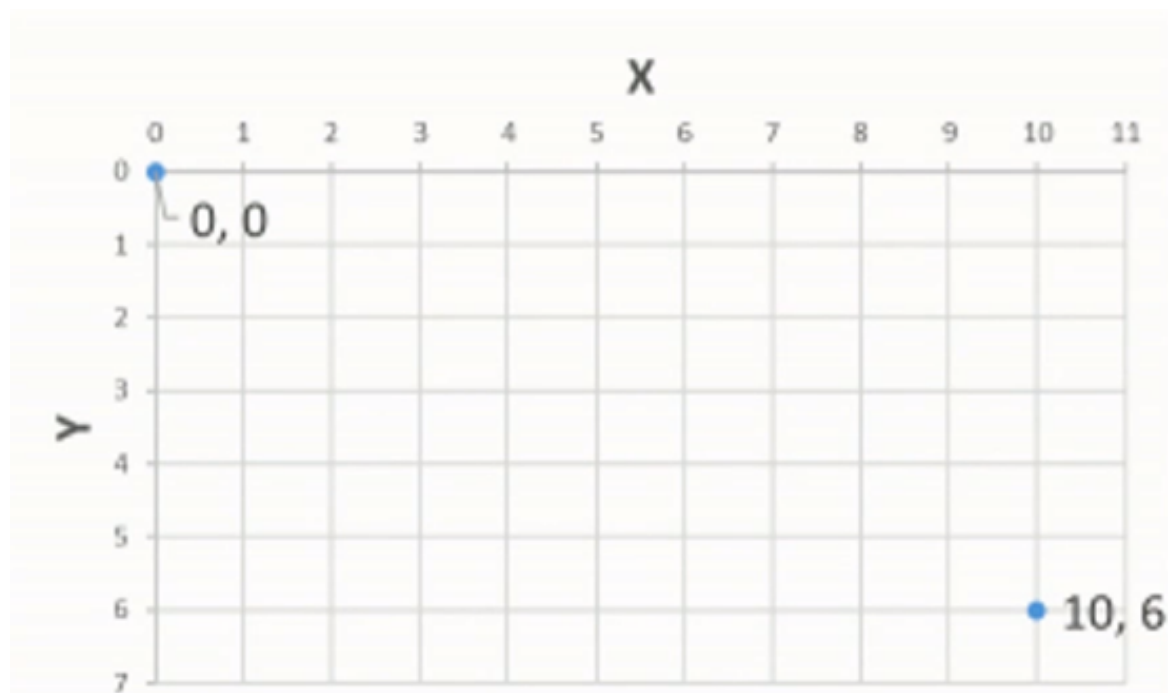
Learning Summary



Preloading assets

```
// Load assets
gameScene.preload = function(){
    // Load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
};
```

- Sprites
 - Game characters and elements.
 - Origin: by default, the middle.
 - Position on x, y can be accessed and changed: set position.
- Coordinate system
 - X: Positive to the right, negative to the left.
 - Y: Positive upwards, negative downwards.



- Challenge
 - Place the sprite in the middle of the screen by changing the position.
 - Don't change the origin.
 - Use the setPosition method.

Some important corrections to the content of this video:

- 1) Scaling a sprite by 2 does NOT double its size, it doubles its width and its height. If you double both the width and the height, the area after scaling is 4x the initial area!
- 2) Similarly, if you scale a sprite by 0.5 you are NOT reducing its area by half — you are reducing both the width and the height by half. This means, the total area is multiplied by 0.25, so a quarter of the original area!

Learning Goals

- Sprite depth
- Changing the size of a sprite.
- Flipping a sprite on X or Y.

We are now familiar with how to position sprites on the screen, as we learned that in the previous lesson.

In this lesson you will learn how to increase or reduce the size of our sprites. This is called **scaling**. You will also learn how to flip sprites, and also what the depth of the sprite does when you render multiple sprites.

We will begin coding by adding a second sprite, which will be the player. See the code below and follow along:

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
  this.load.image('enemy', 'assets/dragon.png');
};

// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  bg.setOrigin(0,0);

  // create the player
  let player = this.add.sprite(70, 180, 'player');

  // we are reducing the width by 50%, and we are doubling the height
  player.setScale(0.5);

  // create an enemy
  let enemy1 = this.add.sprite(250, 180, 'enemy');
```




```
enemy1.scaleX = 2;
enemy1.scaleY = 2;

// create a second enemy
let enemy2 = this.add.sprite(450, 180, 'enemy');
enemy2.displayWidth = 300;

// flip
enemy1.flipX = true;
enemy1.flipY = true;
};

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  width: 640,
  height: 360,
  scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

By default sprites are shown in order, this order is in which they are added. Since we added the background first and then added the player next. The player is showing on top of the background. You can change this by changing the depth of the sprites. The higher the number, the more on top the sprite will be.

When you have sprites and you want to have control over what goes on top you can use the **depth property**.

In order to scale sprites you can use the **set scale method** which works in a similar way to set origin. This means we will have a value for both X and Y. Increase (>1) or reduce (<1) the size of sprites, on X and/or Y. displayWidth and displayHeight: rendered size of the sprite.



```
// create the player
let player = this.add.sprite(70, 180, 'player');

// we are reducing the width by 50%, and we are doubling the height
player.setScale(0.5, 2);

// create an enemy
let enemy1 = this.add.sprite(250, 180, 'enemy');

enemy1.scaleX = 2;
enemy1.scaleY = 2;

// create a second enemy
let enemy2 = this.add.sprite(450, 180, 'enemy');
enemy2.displayWidth = 300;
```

You can **flip a sprite** for both X and Y axes.

```
// flip
enemy1.flipX = true;
enemy1.flipY = true;
```

Learning Summary

- Sprite depth
 - by default, sprites are stacked in the order that are created.
 - The depth property can be used to customize which ones go on top.
 - Sprites with a higher depth value are shown above those with a lower value.
- Scaling
 - Increase (>1) or reduce (<1) the size of sprites, on X and/or Y.
 - displayWidth and displayHeight: rendered size of the sprite.



```
// create the player
let player = this.add.sprite(70, 180, 'player');

// we are reducing the width by 50%, and we are doubling the height
player.setScale(0.5, 2);

// create an enemy
let enemy1 = this.add.sprite(250, 180, 'enemy');

enemy1.scaleX = 2;
enemy1.scaleY = 2;

// create a second enemy
let enemy2 = this.add.sprite(450, 180, 'enemy');
enemy2.displayWidth = 300;
```

- Flipping
 - Can be done for both X and Y.

```
// flip
enemy1.flipX = true;
enemy1.flipY = true;
```



Learning Goals

- Rotating a sprite in angles and radians.
- Update method.
- Changing a property on each frame.

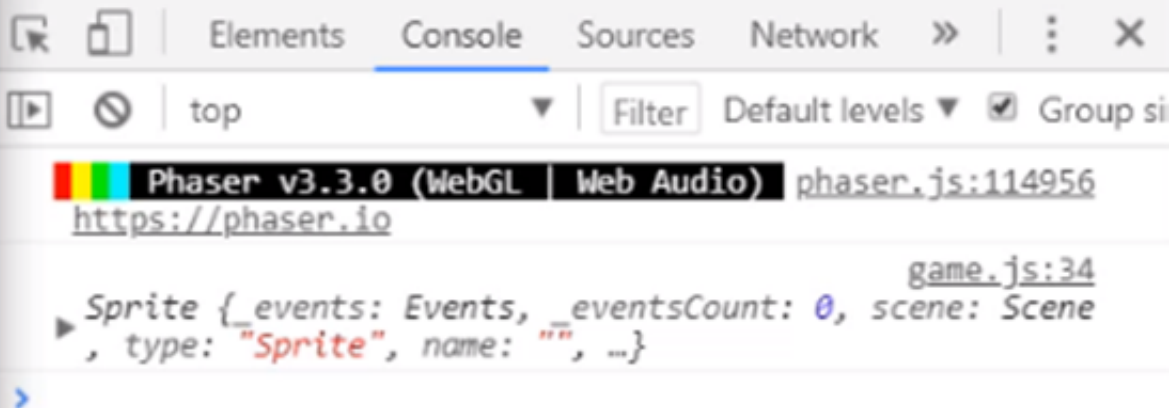
In this lesson you will learn about the **rotation of sprites and the update method**. The update method is part of the scenes life cycle and it is a method that we can use to change the property of something over time.

There are two ways of rotating a sprite. The rotation always takes place about its origins, so the position of the origin doesn't change.

We will begin by increasing the size of the enemy a little bit, see the code below and follow along:

```
// create an enemy
this.enemy1 = this.add.sprite(250, 180, 'enemy');
this.enemy1.setScale(3);

this.enemy1.angle = 45;
this.enemy1.setAngle(45);
```

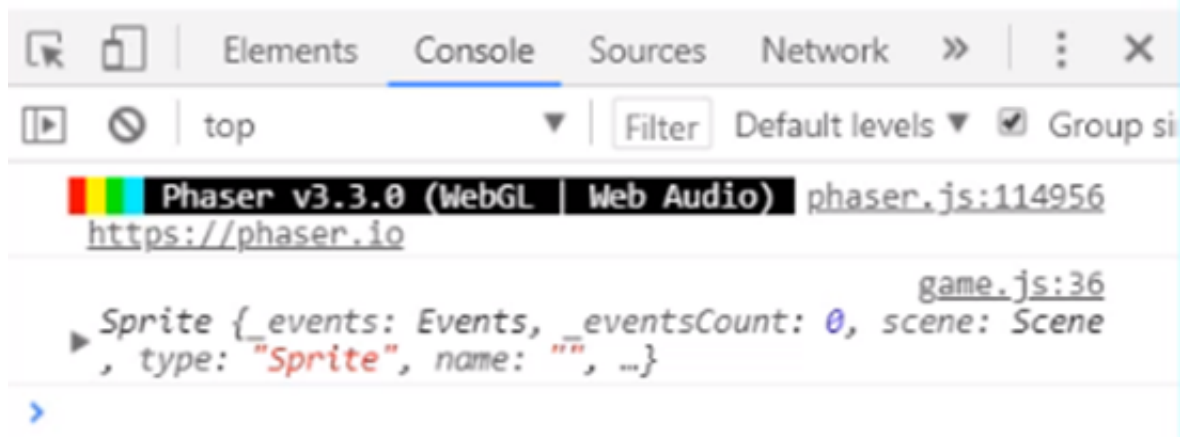


This is how you rotate a sprite by **picking the angles option**.

See the code below and follow along with **how to use the radians option to rotate the sprite**:

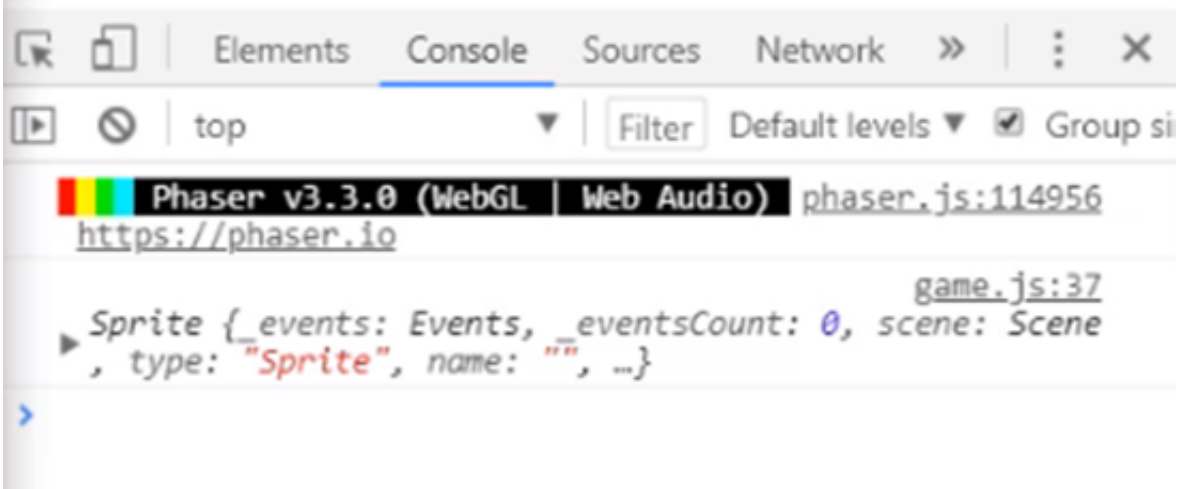
```
this.enemy1.rotation = Math.PI / 4;  
this.enemy1.setRotation(Math.PI / 4);
```

You can see that we are rotating about the origin, which is in the middle.



Now if we didn't want to rotate about the middle anymore, and rotate about a specific point we could do that, see the code below and follow along:

```
this.enemy1.setOrigin(0, 0);
this.enemy1.rotation = Math.PI / 4;
this.enemy1.setRotation(Math.PI / 4);
```

This is how you can rotate sprites and now you know how to change a sprites position, how to scale them up and down, flip sprites, and you now know how to rotate them.

The next thing to cover in this lesson is the **update method**.

In a previous lesson we went over how scenes had special methods.



The **update() method** is called 60 times per second if that is of course supported by the device.



60 frames per second(fps) is the frame rate that the framework aims to have.

We will now create the update method, see the code below and follow along:

```
// this is called up to 60 times per second
gameScene.update = function(){
    //this.enemy1.x += 1;

    this.enemy1.angle += 1;
```

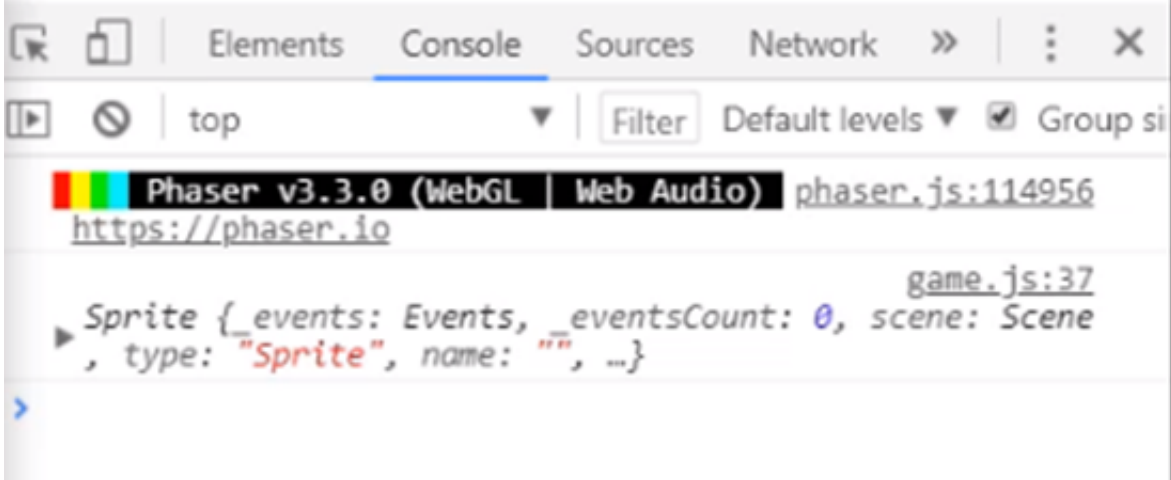
Now there is a challenge for you, and it has to do with playing with the update method.

Challenge- Make a sprite grow over time, only until it reaches twice it's original dimensions(width and height)

Try to solve the challenge on your own without looking at the solution, but if you need help feel free to watch the solution and follow along.

Solving the challenge:

```
// check if we've reached scale of 2
if(this.player.scaleX < 2) {
    // make the player grow
    this.player.scaleX += 0.01;
    this.player.scaleY += 0.01;
}
```

Learning Summary

- Rotation
 - Rotation using your left hand is positive.
 - In angles: `mySprite.angle = 45;`
 - In radians: `mySprite.rotation = Math.PI/4;`





- `update()`
 - Called once up to 60 times per second.
 - Used for things that need to be done or checked “at all times”
 - Executed for the first time after `create()`
- Challenge
 - Make a sprite grow over time, only until it reaches twice it’s original dimensions(width and height)

Learning Goals

- Detecting user input.
- Moving the player.
- Scene init method.
- Checking if two sprites overlap.
- Restarting a scene.

It is now time to implement the logic of the player. We will set up the ability for the player to move in this lesson. The player will be able to **move in the positive X direction**, and we will be adding the goal into the game as well. The goal will be at the end for the player to reach. The goal will be represented by the treasure chest image and then the player reaches the treasure chest we will restart the scene.

There was some housekeeping done to the **game.js file**. Everything that was inside the update method was deleted. The enemy was also deleted.

All you should have in the game to move forward is the player positioned in the middle of the screen.

See the screenshot below:



We want to **detect input in order to make the player move in the game**. There are a few built in ways to do this in Phaser 3.

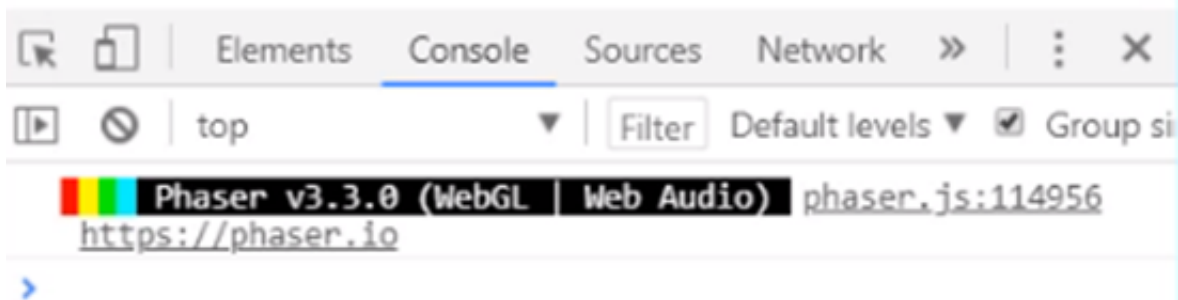
See the code below and follow along:

```
// initiate scene parameters
gameScene.init = function() {
    // player speed
    this.playerSpeed = 5;
};

// check for active input (left click / touch)
```

```
if(this.input.activePointer.isDown) {  
    // player walks  
    this.player.x += this.playerSpeed;  
}
```

Refresh the page and test out the changes.



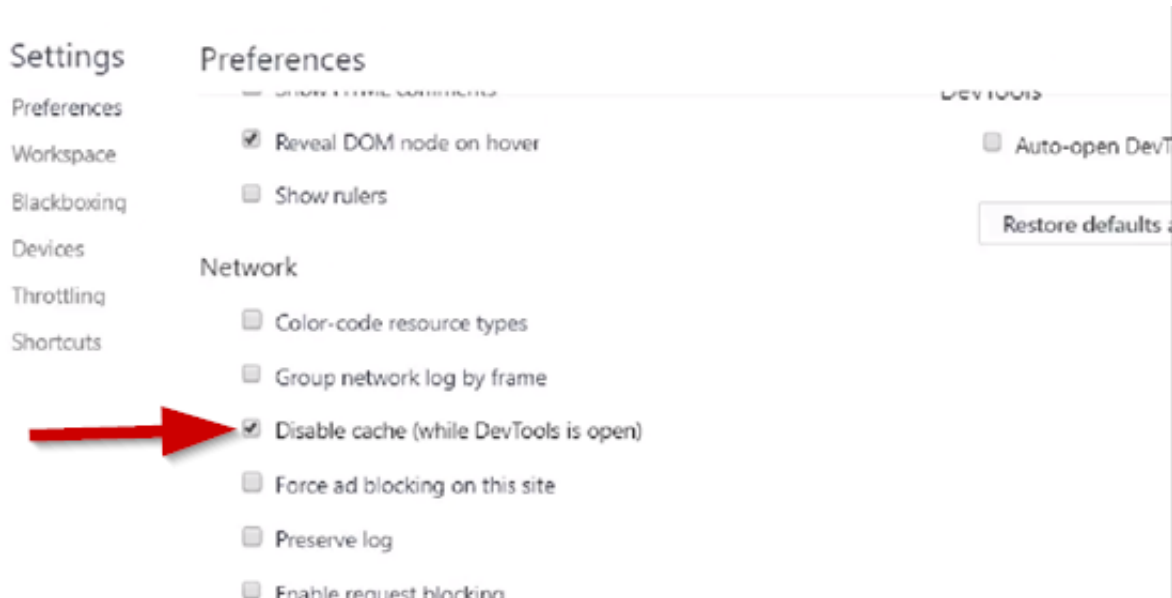
The player is moving quite fast actually. We can change the **playerSpeed** from 5 to 3.

```
// initiate scene parameters  
gameScene.init = function() {  
    // player speed  
    this.playerSpeed = 3;  
};
```

We now have the player moving and moving at a decent speed.

You can also test this for touch screen, you can use the **Chrome Developer Tools** and use the **device tools** to test the game using touch.

It's also important to make sure that the option **"Disable cache(while dev tools are open)"** is checked.



What we want to do now is add in the treasure chest to the game. We want to detect collision or overlapping of different elements when the player reaches that point.

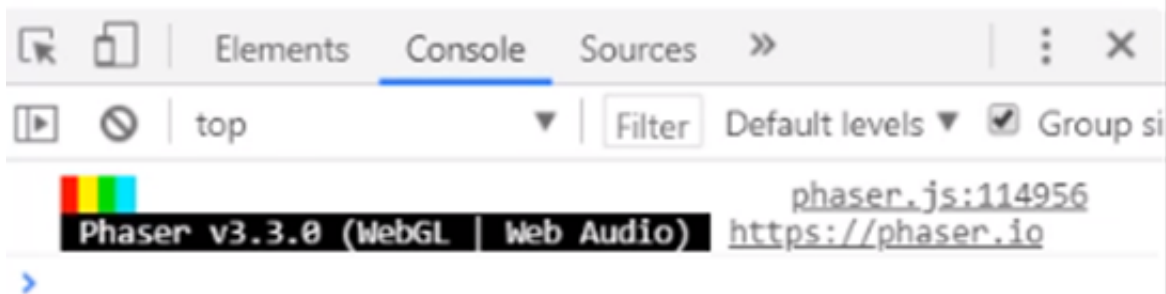
We need to first **pre-load** the sprite. See the code below and follow along:

```
// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
  this.load.image('enemy', 'assets/dragon.png');
  this.load.image('goal', 'assets/treasure.png');
};
```

Now we need to **create a new sprite for that element** we just added above and change the scale of the treasure chest. See the code below and follow along:

```
// goal
this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.height / 2, 'goal');
this.goal.setScale(0.6);
```


Refresh the page.



What we want to do now is **check for collision with the player and the treasure chest**.

```
// treasure overlap check
let playerRect = this.player.getBounds();
let treasureRect = this.goal.getBounds();
```

```
if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
    console.log('reached goal!');

    // restart the Scene
    this.scene.restart();

    // make sure we leave this method
    return;
}

};
```

Refresh the page and test out the changes.

The player should now move and when the player collides with the treasure chest the scene is restarted.

Here is the entire **game.js** file, if you are having issues you can compare your code to this:

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// initiate scene parameters
gameScene.init = function() {
    // player speed
    this.playerSpeed = 3;
};

// load assets
gameScene.preload = function(){
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
    this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
    // create bg sprite
    let bg = this.add.sprite(0, 0, 'background');

    // change the origin to the top-left corner
    bg.setOrigin(0,0);

    // create the player
    this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

    // we are reducing the width and height by 50%
    this.player.setScale(0.5);

    // goal
    this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.h
```



```
eight / 2, 'goal');
    this.goal.setScale(0.6);
};

// this is called up to 60 times per second
gameScene.update = function(){

    // check for active input (left click / touch)
    if(this.input.activePointer.isDown) {
        // player walks
        this.player.x += this.playerSpeed;
    }

    // treasure overlap check
    let playerRect = this.player.getBounds();
    let treasureRect = this.goal.getBounds();

    if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
        console.log('reached goal!');

        // restart the Scene
        this.scene.restart();

        // make sure we leave this method
        return;
    }
};

// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Learning Summary

- Detecting user input
 - There are many ways to do this in Phaser.
 - We are accessing the scene's input object and check for the active input:



```
// check for active input (left click / touch)  
if(this.input.activePointer.isDown) {  
    // player walks  
    this.player.x += this.playerSpeed;  
}
```

- Init method
 - Used to set scene parameters, it runs before preload..

```
// initiate scene parameters  
gameScene.init = function() {  
    // player speed  
    this.playerSpeed = 3;  
};
```

- Checking if two sprites overlap
 - Geometrical utility to check if two rectangles overlap.
 - This can also be done using a physics engine (overkill for this game)

```
// treasure overlap check  
let playerRect = this.player.getBounds();  
let treasureRect = this.goal.getBounds();  
  
if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {  
    console.log('reached goal!');  
}
```

- Restarting a scene
 - Tell the Scene Manager to reboot the current scene.

```
// restart the Scene  
this.scene.manager.bootScene(this);
```



Learning Goals

- Creating a bouncing enemy.
- Assigning a random direction and speed.

We will be adding in a single enemy to our game that moves up and down.

We will **begin by adding the sprite for the dragon**. We already have the image loaded in the file. See the code below and follow along:

```
// enemy
this.enemy = this.add.sprite(120, this.sys.game.config.height / 2, 'enemy');
this.enemy.flipX = true;
this.enemy.setScale(0.6);
```



We will now **make the enemy move up and down**. We will start by adding a speed for the enemy. See the code below and follow along:

```
// enemy speed
this.enemySpeed = 3;

// enemy movement
this.enemy.y += this.enemy.speed;
```

Refresh the page.



The enemy moves down, but not back up, and always at the same speed. The enemy isn't bouncing currently.

We can now **implement the bouncing speed**. See the code below and follow along:

```
// boundaries
this.enemyMinY = 80;
this.enemyMaxY = 280;

// check we haven't passed min or max Y
let conditionUp = this.enemy.speed < 0 && this.enemy.y <= this.enemyMinY;
let conditionDown = this.enemy.speed > 0 && this.enemy.y >= this.enemyMaxY;

// if we passed the upper or lower limit, reverse
if(conditionUp || conditionDown) {
    this.enemy.speed *= -1;
}
```

We now have a bouncy enemy dragon and its always moving at the same speed. We will be adding in more enemy dragons in the next lesson, but we will not want them all moving at the same speed. We also don't want the dragons moving in the same direction at the same time, we want to add some randomization to the movement and speed to the enemy dragons.

We will now **setup the random enemy speed**. See the code below and follow along:

```
// enemy speed
this.enemyMinSpeed = 2;
this.enemyMaxSpeed = 4.5;

// set enemy speed
let dir = Math.random() < 0.5 ? 1 : -1;
```



```
let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
this.enemy.speed = dir * speed;
```

Refresh the page a few times and you will see that the dragon moves at different speeds each time you refresh the page.

Here is the entire **game.js** file if you are having issues with your code you can compare it to this:

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// initiate scene parameters
gameScene.init = function() {
  // player speed
  this.playerSpeed = 3;

  // enemy speed
  this.enemyMinSpeed = 2;
  this.enemyMaxSpeed = 4.5;

  // boundaries
  this.enemyMinY = 80;
  this.enemyMaxY = 280;
};

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
  this.load.image('enemy', 'assets/dragon.png');
  this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  bg.setOrigin(0,0);

  // create the player
  this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

  // we are reducing the width and height by 50%
  this.player.setScale(0.5);

  // goal
  this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.height / 2, 'goal');
  this.goal.setScale(0.6);
```



```
// enemy
this.enemy = this.add.sprite(120, this.sys.game.config.height / 2, 'enemy');
this.enemy.flipX = true;
this.enemy.setScale(0.6);

// set enemy speed
let dir = Math.random() < 0.5 ? 1 : -1;
let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
this.enemy.speed = dir * speed;
};

// this is called up to 60 times per second
gameScene.update = function(){

    // check for active input (left click / touch)
    if(this.input.activePointer.isDown) {
        // player walks
        this.player.x += this.playerSpeed;
    }

    // treasure overlap check
    let playerRect = this.player.getBounds();
    let treasureRect = this.goal.getBounds();

    if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
        console.log('reached goal!');

        // restart the Scene
        this.scene.restart();
        return;
    }

    // enemy movement
    this.enemy.y += this.enemy.speed;

    // check we haven't passed min or max Y
    let conditionUp = this.enemy.speed < 0 && this.enemy.y <= this.enemyMinY;
    let conditionDown = this.enemy.speed > 0 && this.enemy.y >= this.enemyMaxY;

    // if we passed the upper or lower limit, reverse
    if(conditionUp || conditionDown) {
        this.enemy.speed *= -1;
    }
};

// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: gameScene
};
```



```
// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Learning Summary

- Bouncing movement
 - Move the enemy
 - Check if it's reached any of the limits.
 - If it's reached a limit, and it was moving in that direction, reverse speed.

```
// check we haven't passed min or max Y
let conditionUp = this.enemy.speed < 0 && this.enemy.y <= this.enemyMinY;
let conditionDown = this.enemy.speed > 0 && this.enemy.y >= this.enemyMaxY;
```

- Random velocity
 - Direction

```
let dir = Math.random() < 0.5 ? 1 : -1;
```

- Speed

```
let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
```




Learning Goals

- Groups in Phaser
- Creating multiple sprites in a group.
- Applying a method to all group elements.
- Implementing the enemies group.

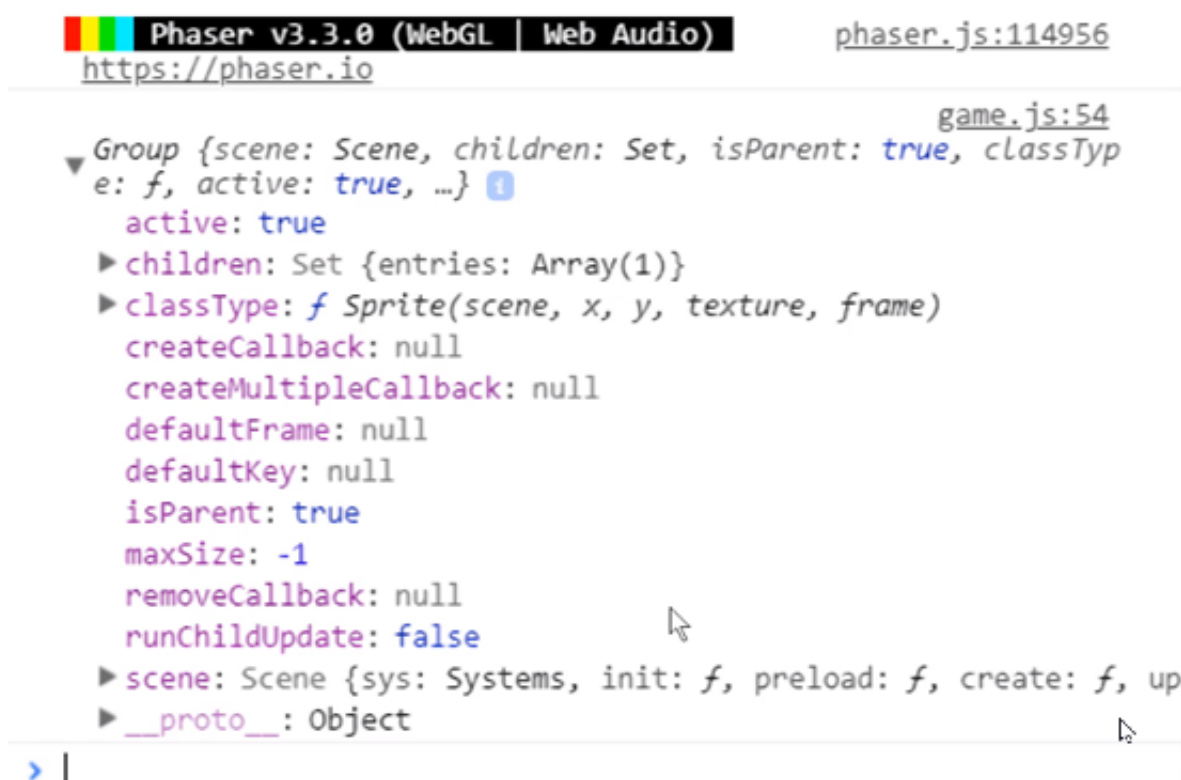
In this lesson you will be introduced to the concept of **groups**. You can think of a group as a collection of sprites.

We will begin by **creating a group for our enemies**. See the code below and follow along:

```
//enemy
this.enemies = this.add.group();

this.enemy = this.add.sprite(120, this.sys.game.config.height / 2, 'enemy');
this.enemy.flipX = true;
this.enemy.setScale(0.6);
```

Refresh the page and look at the Console window:



We will be able to access our elements at any time by simply doing the **get children method**.

See the code below and follow along:

```
console.log(this.enemies.getChildren());
```



You can see that you are given a normal array with all the sprite elements.

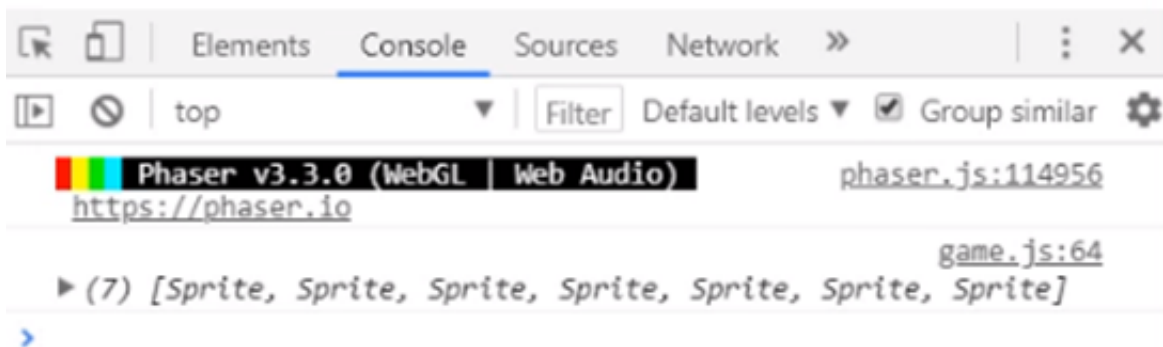
You can **use groups to change the scale of your elements**.

You can pass an object to the group declaration and that object can contain how many elements we want to create and what sprites are seen.

See the code below and follow along:

```
// enemy group
this.enemies = this.add.group({
  key: 'enemy',
  repeat: 5,
  setXY: {
    x: 110,
    y: 100,
    stepX: 80,
    stepY: 20
  }
})
```

Refresh the page.



So we now have a bunch of sprites that are separated and the previous enemy is bouncing up and down.

Now we need to be able to move the other dragons and then flip them. See the code below and follow along:

```
// enemy group
this.enemies = this.add.group({
  key: 'enemy',
```



```
        repeat: 5,
        setXY: {
            x: 90,
            y: 100,
            stepX: 80,
            stepY: 20
        }
    });

    // setting scale to all group elements
    Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);

    // set flipX, and speed
    Phaser.Actions.Call(this.enemies.getChildren(), function(enemy){
        // flip enemy
        enemy.flipX = true;

        // set speed
        let dir = Math.random() < 0.5 ? 1 : -1;
        let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
        enemy.speed = dir * speed;

    }, this);
};
```

Make sure you have deleted the previous created enemy.

We can now setup the movement. See the code below and follow along:

```
// get enemies
let enemies = this.enemies.getChildren();
let numEnemies = enemies.length;

for(let i = 0; i < numEnemies; i++) {

    // enemy movement
    enemies[i].y += enemies[i].speed;

    // check we haven't passed min or max Y
    let conditionUp = enemies[i].speed < 0 && enemies[i].y <= this.enemyMinY;
    let conditionDown = enemies[i].speed > 0 && enemies[i].y >= this.enemyMaxY;

    // if we passed the upper or lower limit, reverse
    if(conditionUp || conditionDown) {
        enemies[i].speed *= -1;
    }
}
```

Refresh the page.



We now have all of our enemies moving at random speeds.

There is one thing missing though with the game and that is the collision checking with the enemies and the player.

See the code below and follow along:

```
// check enemy overlap
let enemyRect = enemies[i].getBounds();

if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
    console.log('Game over!');

    // restart the Scene
    this.scene.restart();
    return;
}
```

Refresh the page and test out the changes. You will now see that the collision check is indeed working. Every time that player collides with an enemy the scene is restarted.

Feel free to adapt the game to make it easier if you wish.

What we need to do in the next lesson is take care of the game over implementation.

Here is the update **game.js** file for you to compare yours to if you are having issues:

```
// create a new scene
let gameScene = new Phaser.Scene('Game');
```




```
// initiate scene parameters
gameScene.init = function() {
  // player speed
  this.playerSpeed = 3;

  // enemy speed
  this.enemyMinSpeed = 2;
  this.enemyMaxSpeed = 4.5;

  // boundaries
  this.enemyMinY = 80;
  this.enemyMaxY = 280;
};

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
  this.load.image('enemy', 'assets/dragon.png');
  this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  bg.setOrigin(0,0);

  // create the player
  this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

  // we are reducing the width and height by 50%
  this.player.setScale(0.5);

  // goal
  this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.height / 2, 'goal');
  this.goal.setScale(0.6);

  // enemy group
  this.enemies = this.add.group({
    key: 'enemy',
    repeat: 5,
    setXY: {
      x: 90,
      y: 100,
      stepX: 80,
      stepY: 20
    }
  });

  // setting scale to all group elements
```




```
Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);

// set flipX, and speed
Phaser.Actions.Call(this.enemies.getChildren(), function(enemy){
    // flip enemy
    enemy.flipX = true;

    // set speed
    let dir = Math.random() < 0.5 ? 1 : -1;
    let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
    enemy.speed = dir * speed;

}, this);
};

// this is called up to 60 times per second
gameScene.update = function(){

    // check for active input (left click / touch)
    if(this.input.activePointer.isDown) {
        // player walks
        this.player.x += this.playerSpeed;
    }

    // treasure overlap check
    let playerRect = this.player.getBounds();
    let treasureRect = this.goal.getBounds();

    if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
        console.log('reached goal!');

        // restart the Scene
        this.scene.restart();
        return;
    }

    // get enemies
    let enemies = this.enemies.getChildren();
    let numEnemies = enemies.length;

    for(let i = 0; i < numEnemies; i++) {

        // enemy movement
        enemies[i].y += enemies[i].speed;

        // check we haven't passed min or max Y
        let conditionUp = enemies[i].speed < 0 && enemies[i].y <= this.enemyMinY;
        let conditionDown = enemies[i].speed > 0 && enemies[i].y >= this.enemyMaxY;

        // if we passed the upper or lower limit, reverse
        if(conditionUp || conditionDown) {
            enemies[i].speed *= -1;
        }
    }
}
```



```
// check enemy overlap
let enemyRect = enemies[i].getBounds();

if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
    console.log('Game over!');

    // restart the Scene
    this.scene.restart();
    return;
}
}

};

// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Learning Summary

- Groups
 - Collection of sprites
 - Ability to create multiple sprites
 - Ability to apply specific methods to all sprites
 - Groups don't have X, Y coordinates. Member sprites do

```
// enemy group
this.enemies = this.add.group({
  key: 'enemy',
  repeat: 5,
  setXY: {
    x: 90,
    y: 100,
    stepX: 80,
    stepY: 20
  }
});
```

- Calling a custom method

```
Phaser.Actions.Call(this.enemies.getChildren(), function(item){
  // method logic here
}, this);
```

Learning Goals

- Camera effects: Shake and fade
- Chaining effects
- Event listening

In this lesson we will be adding some nice **camera effects** to our game. We will be using events for this, and you will learn about how to use events in Phaser as well.

Currently all we are doing is restarting the scene once we collide with an enemy or the player reaches the treasure.

We need to move our restart scene code into a separate method. See the code below and follow along:

```
// end game
return this.gameOver();
```

We can now begin working on the camera shake effect. See the code below and follow along:

```
gameScene.gameOver = function() {

// shake camera
this.cameras.main.shake(500);
```

Refresh the page and test out the changes.

You should now see the camera shake effect when the player collides with an enemy. What we want to do now is restart the scene once the camera shake effect completes.

See the code below and follow along:

```
gameScene.gameOver = function() {

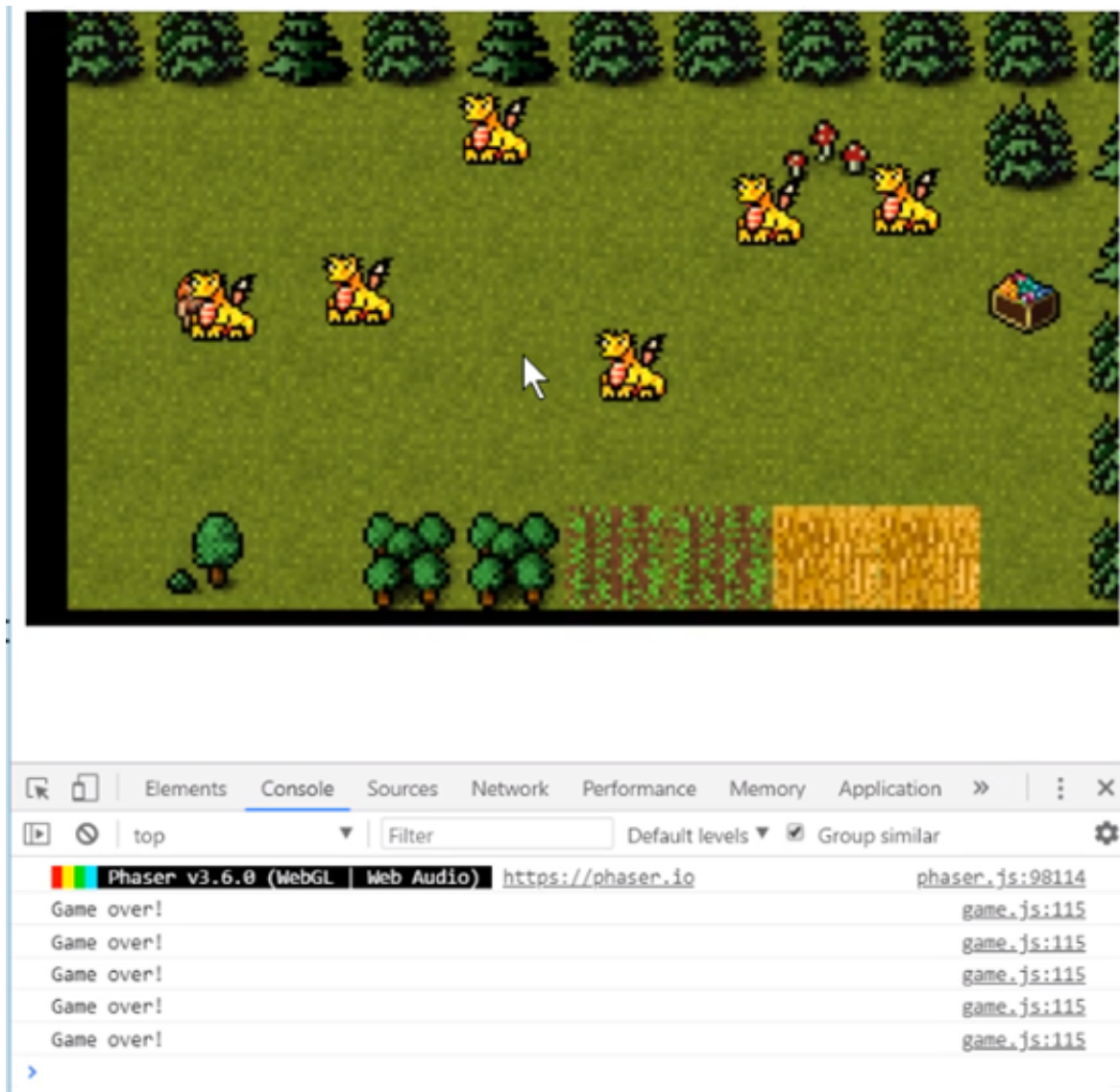
    // shake camera
    this.cameras.main.shake(500);

    // listen for event completion
    this.cameras.main.on('camerashakecomplete', function(camera, effect){

        // restart the Scene
        this.scene.restart();
    }, this);

};
```

Refresh the page and test the changes out.



So now when we collide with something the **Game Over** is being called but then while the camera event is taking place, we are calling the camera shake over and over again.

We don't really want this so we need to have some sort of flag that helps us call it just once.

We can create a **boolean variable** and take care of this pretty easily. See the code below and follow along:

```
// we are not terminating
this.isTerminating = false;

gameScene.gameOver = function() {

    // initiated game over sequence
    this.isTerminating = true;

    // shake camera
    this.cameras.main.shake(500);
```

```
// listen for event completion
this.cameras.main.on('camerashakecomplete', function(camera, effect){

});

// don't execute if we are terminating
if(this.isTerminating) return;
```

Refresh the page and test the changes. You can now see that game over is happening once now.

We can now add the **camera fade** to the game. see the code below and follow along:

```
// fade out
this.cameras.main.fade(500);
}, this);

this.cameras.main.on('camerafadeoutcomplete', function(camera, effect){
  // restart the Scene
  this.scene.restart();
}, this);
```

Refresh the page and you will see that once the camera shakes the scene will fade.

Here is the entire final version of the **game.js** file:

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// initiate scene parameters
gameScene.init = function() {
  // player speed
  this.playerSpeed = 3;

  // enemy speed
  this.enemyMinSpeed = 2;
  this.enemyMaxSpeed = 4.5;

  // boundaries
  this.enemyMinY = 80;
  this.enemyMaxY = 280;

  // we are not terminating
  this.isTerminating = false;
};

// load assets
gameScene.preload = function(){
```



```
// load images
this.load.image('background', 'assets/background.png');
this.load.image('player', 'assets/player.png');
this.load.image('enemy', 'assets/dragon.png');
this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
    // create bg sprite
    let bg = this.add.sprite(0, 0, 'background');

    // change the origin to the top-left corner
    bg.setOrigin(0,0);

    // create the player
    this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

    // we are reducing the width and height by 50%
    this.player.setScale(0.5);

    // goal
    this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.height / 2, 'goal');
    this.goal.setScale(0.6);

    // enemy group
    this.enemies = this.add.group({
        key: 'enemy',
        repeat: 5,
        setXY: {
            x: 90,
            y: 100,
            stepX: 80,
            stepY: 20
        }
    });

    // setting scale to all group elements
    Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);

    // set flipX, and speed
    Phaser.Actions.Call(this.enemies.getChildren(), function(enemy){
        // flip enemy
        enemy.flipX = true;

        // set speed
        let dir = Math.random() < 0.5 ? 1 : -1;
        let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
        enemy.speed = dir * speed;

    }, this);
};
```




```
// this is called up to 60 times per second
gameScene.update = function(){

    // don't execute if we are terminating
    if(this.isTerminating) return;

    // check for active input (left click / touch)
    if(this.input.activePointer.isDown) {
        // player walks
        this.player.x += this.playerSpeed;
    }

    // treasure overlap check
    let playerRect = this.player.getBounds();
    let treasureRect = this.goal.getBounds();

    if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
        console.log('reached goal!');

        // end game
        return this.gameOver();
    }

    // get enemies
    let enemies = this.enemies.getChildren();
    let numEnemies = enemies.length;

    for(let i = 0; i< numEnemies; i++) {

        // enemy movement
        enemies[i].y += enemies[i].speed;

        // check we haven't passed min or max Y
        let conditionUp = enemies[i].speed < 0 && enemies[i].y <= this.enemyMinY;
        let conditionDown = enemies[i].speed > 0 && enemies[i].y >= this.enemyMaxY;

        // if we passed the upper or lower limit, reverse
        if(conditionUp || conditionDown) {
            enemies[i].speed *= -1;
        }

        // check enemy overlap
        let enemyRect = enemies[i].getBounds();

        if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
            console.log('Game over!');

            // end game
            return this.gameOver();
        }
    }
};

gameScene.gameOver = function() {
```



```
// initiated game over sequence
this.isTerminating = true;

// shake camera
this.cameras.main.shake(500);

// listen for event completion
this.cameras.main.on('camerashakecomplete', function(camera, effect){

    // fade out
    this.cameras.main.fade(500);
}, this);

this.cameras.main.on('camerafadeoutcomplete', function(camera, effect){
    // restart the Scene
    this.scene.restart();
}, this);

};

// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Learning Summary

- Camera effects
 - Access main camera: `this.camera.main`
 - Built-in events: fade, shake, flash
 - Listening for completion:

```
// shake camera
this.cameras.main.shake(500);

// listen for event completion
this.cameras.main.on('camerashakecomplete', function(camera, effect){

    // fade out
    this.cameras.main.fade(500);
```

Add a Title Screen

We can add a title screen to our games by adding an additional **Phaser.Scene**.

Adding text to introduce the game and then a click function to advance the player to the gameplay are all that are required, besides having a nice-looking background image.

Let's go ahead and add a title screen to our road crossing game now by adding the following code to the top of **game.js**:

```
let titleScene = new Phaser.Scene('Title');

titleScene.preload = function(){
    this.load.image('titlebackground', 'assets/title-background.png');
}

titleScene.create = function () {
    let backgr = this.add.sprite(0,0, 'titlebackground');
    backgr.setOrigin(0,0)

    let text = this.add.text(100,100, 'Welcome to my Game');

    this.input.on('pointerup', function (pointer) {
        this.scene.start('Game');
    }, this);
}
```

Then, at the bottom of **game.js**, update the **config** section to include the **titleScene** in the scene array:

```
// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: [titleScene, gameScene]
};
```

That's it!

You can use the below image as your background or create your own!



If you create your own, be sure to use 640 x 360 pixels for the dimensions of your image and save it as a **PNG** file.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Starting

index.html

Found in Project/00 - Starting

This code template creates a basic HTML page and includes a script tag intended for linking an external JavaScript file. It is commonly used in web development for setting up interactive content or applications.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Learn Game Development at Zenva.com</title>
</head>
<body>
  <script src=""></script>
</body>
</html>
```

Hello World Phaser

index.html

Found in Project/01 - Hello World Phaser

This is an HTML skeleton for a Phaser-based web game. The canvas for the game is set to take up the full width of the page. It loads two JavaScript files, phaser.js (the Phaser game framework library) and game.js (the main game file).

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Learn Game Development at Zenva.com</title>
  <style>
    canvas {
      width: 100%;
    }
  </style>
</head>
<body>
  <script src="js/phaser.js"></script>
  <script src="js/game.js"></script>
</body>
```

</html>

game.js

Found in Project/01 - Hello World Phaser/js

This code sets up a new Phaser game by creating a scene, defining the game configuration (type, width, height, scene), and initializing the game with the specified configuration.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  width: 640,
  height: 360,
  scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Rendering Sprites

game.js

Found in Project/02 - Rendering Sprites/js

This Phaser game development code sets up a scene with functions to preload assets (images) and create sprites. It also configures the game dimensions and initializes the game for rendering with the specified scene.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
};

// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  //bg.setOrigin(0,0);
```

```
// place sprite in the center
bg.setPosition(640/2, 360/2);

let gameW = this.sys.game.config.width;
let gameH = this.sys.game.config.height;

console.log(gameW, gameH);

console.log(bg);
console.log(this);
};

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  width: 640,
  height: 360,
  scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Scaling and Flipping

game.js

Found in Project/03 - Scaling and Flipping/js

This Phaser game development code sets up a scene with functions to preload images and create sprites for the background, player, and enemies. It includes scaling, positioning, and flipping properties for the sprites along with configuring the game dimensions and initializing the game for rendering.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
  this.load.image('enemy', 'assets/dragon.png');
};

// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  bg.setOrigin(0,0);
```

```
// create the player
let player = this.add.sprite(70, 180, 'player');

// we are reducing the width by 50%, and we are doubling the height
player.setScale(0.5);

// create an enemy
let enemy1 = this.add.sprite(250, 180, 'enemy');

enemy1.scaleX = 2;
enemy1.scaleY = 2;

// create a second enemy
let enemy2 = this.add.sprite(450, 180, 'enemy');
enemy2.displayWidth = 300;

// flip
enemy1.flipX = true;
enemy1.flipY = true;
};

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  width: 640,
  height: 360,
  scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Rotation and Update

game.js

Found in Project/04 - Rotation and Update/js

This Phaser game development code sets up a scene to load images, create sprites for the background, player, and enemy, and configure their properties like scaling and rotation. It also includes an update function to animate the sprites and initializes the game with the specified scene and configuration.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
  this.load.image('enemy', 'assets/dragon.png');
```



```
};

// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  bg.setOrigin(0,0);

  // create the player
  this.player = this.add.sprite(70, 180, 'player');

  // we are reducing the width and height by 50%
  this.player.setScale(0.5);

  // create an enemy
  this.enemy1 = this.add.sprite(250, 180, 'enemy');
  this.enemy1.setScale(3);

  //this.enemy1.angle = 45;
  //this.enemy1.setAngle(45);

  //this.enemy1.setOrigin(0, 0);
  this.enemy1.rotation = Math.PI / 4;
  this.enemy1.setRotation(Math.PI / 4);

  console.log(this.enemy1);
};

// this is called up to 60 times per second
gameScene.update = function(){
  //this.enemy1.x += 1;

  this.enemy1.angle += 1;

  // check if we've reached scale of 2
  if(this.player.scaleX < 2) {
    // make the player grow
    this.player.scaleX += 0.01;
    this.player.scaleY += 0.01;
  }
};

// set the configuration of the game
let config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  width: 640,
  height: 360,
  scene: gameScene
};

// create a new game, pass the configuration
```

```
let game = new Phaser.Game(config);
```

Player

game.js

Found in Project/05 - Player/js

This Phaser game development code creates a scene where a player sprite can move on input and reach a goal sprite for a simple game scenario. The scene parameters, asset loading, player movement, collision detection with the goal, and scene restarting functionalities are defined within the game setup.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// initiate scene parameters
gameScene.init = function() {
    // player speed
    this.playerSpeed = 3;
};

// load assets
gameScene.preload = function(){
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
    this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
    // create bg sprite
    let bg = this.add.sprite(0, 0, 'background');

    // change the origin to the top-left corner
    bg.setOrigin(0,0);

    // create the player
    this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

    // we are reducing the width and height by 50%
    this.player.setScale(0.5);

    // goal
    this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.h
    eight / 2, 'goal');
    this.goal.setScale(0.6);
};

// this is called up to 60 times per second
gameScene.update = function(){
```

```
// check for active input (left click / touch)
if(this.input.activePointer.isDown) {
    // player walks
    this.player.x += this.playerSpeed;
}

// treasure overlap check
let playerRect = this.player.getBounds();
let treasureRect = this.goal.getBounds();

if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
    console.log('reached goal!');

    // restart the Scene
    this.scene.restart();

    // make sure we leave this method
    return;
}

};

// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Enemy

game.js

Found in Project/06 - Enemy/js

This Phaser game development code sets up a scene where a player can move to reach a goal sprite while avoiding an enemy sprite that moves within specified boundaries and speed. Scene parameters, asset loading, player movement, enemy behavior, collision detection, and scene restarting are defined for an interactive game scenario.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// initiate scene parameters
gameScene.init = function() {
    // player speed
    this.playerSpeed = 3;
```



```
// enemy speed
this.enemyMinSpeed = 2;
this.enemyMaxSpeed = 4.5;

// boundaries
this.enemyMinY = 80;
this.enemyMaxY = 280;
};

// load assets
gameScene.preload = function(){
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
    this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
    // create bg sprite
    let bg = this.add.sprite(0, 0, 'background');

    // change the origin to the top-left corner
    bg.setOrigin(0,0);

    // create the player
    this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

    // we are reducing the width and height by 50%
    this.player.setScale(0.5);

    // goal
    this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.height / 2, 'goal');
    this.goal.setScale(0.6);

    // enemy
    this.enemy = this.add.sprite(120, this.sys.game.config.height / 2, 'enemy');
    this.enemy.flipX = true;
    this.enemy.setScale(0.6);

    // set enemy speed
    let dir = Math.random() < 0.5 ? 1 : -1;
    let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
    this.enemy.speed = dir * speed;
};

// this is called up to 60 times per second
gameScene.update = function(){

    // check for active input (left click / touch)
    if(this.input.activePointer.isDown) {
        // player walks
```

```
this.player.x += this.playerSpeed;
}

// treasure overlap check
let playerRect = this.player.getBounds();
let treasureRect = this.goal.getBounds();

if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
    console.log('reached goal!');

    // restart the Scene
    this.scene.restart();
    return;
}

// enemy movement
this.enemy.y += this.enemy.speed;

// check we haven't passed min or max Y
let conditionUp = this.enemy.speed < 0 && this.enemy.y <= this.enemyMinY;
let conditionDown = this.enemy.speed > 0 && this.enemy.y >= this.enemyMaxY;

// if we passed the upper or lower limit, reverse
if(conditionUp || conditionDown) {
    this.enemy.speed *= -1;
}

};

// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Groups

game.js

Found in Project/07 - Groups/js

This Phaser game development code creates a scene with a player sprite moving towards a goal while avoiding enemy sprites that move within specified boundaries and speeds. It incorporates asset loading, group handling for enemies, movement logic, collision detection, and scene restarting for an interactive game scenario with multiple enemies.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');
```

```
// initiate scene parameters
gameScene.init = function() {
  // player speed
  this.playerSpeed = 3;

  // enemy speed
  this.enemyMinSpeed = 2;
  this.enemyMaxSpeed = 4.5;

  // boundaries
  this.enemyMinY = 80;
  this.enemyMaxY = 280;
};

// load assets
gameScene.preload = function(){
  // load images
  this.load.image('background', 'assets/background.png');
  this.load.image('player', 'assets/player.png');
  this.load.image('enemy', 'assets/dragon.png');
  this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
  // create bg sprite
  let bg = this.add.sprite(0, 0, 'background');

  // change the origin to the top-left corner
  bg.setOrigin(0,0);

  // create the player
  this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

  // we are reducing the width and height by 50%
  this.player.setScale(0.5);

  // goal
  this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.height / 2, 'goal');
  this.goal.setScale(0.6);

  // enemy group
  this.enemies = this.add.group({
    key: 'enemy',
    repeat: 5,
    setXY: {
      x: 90,
      y: 100,
      stepX: 80,
      stepY: 20
    }
  });
};
```



```
// setting scale to all group elements
Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);

// set flipX, and speed
Phaser.Actions.Call(this.enemies.getChildren(), function(enemy){
    // flip enemy
    enemy.flipX = true;

    // set speed
    let dir = Math.random() < 0.5 ? 1 : -1;
    let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemyMinSpeed);
    enemy.speed = dir * speed;

}, this);
};

// this is called up to 60 times per second
gameScene.update = function(){

    // check for active input (left click / touch)
    if(this.input.activePointer.isDown) {
        // player walks
        this.player.x += this.playerSpeed;
    }

    // treasure overlap check
    let playerRect = this.player.getBounds();
    let treasureRect = this.goal.getBounds();

    if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
        console.log('reached goal!');

        // restart the Scene
        this.scene.restart();
        return;
    }

    // get enemies
    let enemies = this.enemies.getChildren();
    let numEnemies = enemies.length;

    for(let i = 0; i < numEnemies; i++) {

        // enemy movement
        enemies[i].y += enemies[i].speed;

        // check we haven't passed min or max Y
        let conditionUp = enemies[i].speed < 0 && enemies[i].y <= this.enemyMinY;
        let conditionDown = enemies[i].speed > 0 && enemies[i].y >= this.enemyMaxY;

        // if we passed the upper or lower limit, reverse
        if(conditionUp || conditionDown) {
            enemies[i].speed *= -1;
        }
    }
}
```

```
// check enemy overlap
let enemyRect = enemies[i].getBounds();

if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
    console.log('Game over!');

    // restart the Scene
    this.scene.restart();
    return;
}
}

};

// set the configuration of the game
let config = {
    type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
    width: 640,
    height: 360,
    scene: gameScene
};

// create a new game, pass the configuration
let game = new Phaser.Game(config);
```

Camera Effects

game.js

Found in Project/08 - Camera Effects/js

This Phaser game code sets up a scene where a player must reach a goal while avoiding enemies, with game over scenarios triggering camera effects and scene restarts. It includes asset loading, enemy group handling, collision detection, implementing a game over sequence and scene restart functionality for an interactive gaming experience.

```
// create a new scene
let gameScene = new Phaser.Scene('Game');

// initiate scene parameters
gameScene.init = function() {
    // player speed
    this.playerSpeed = 3;

    // enemy speed
    this.enemyMinSpeed = 2;
    this.enemyMaxSpeed = 4.5;

    // boundaries
    this.enemyMinY = 80;
    this.enemyMaxY = 280;
```



```
// we are not terminating
this.isTerminating = false;
};

// load assets
gameScene.preload = function(){
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
    this.load.image('goal', 'assets/treasure.png');
};

// called once after the preload ends
gameScene.create = function() {
    // create bg sprite
    let bg = this.add.sprite(0, 0, 'background');

    // change the origin to the top-left corner
    bg.setOrigin(0,0);

    // create the player
    this.player = this.add.sprite(40, this.sys.game.config.height / 2, 'player');

    // we are reducing the width and height by 50%
    this.player.setScale(0.5);

    // goal
    this.goal = this.add.sprite(this.sys.game.config.width - 80, this.sys.game.config.height / 2, 'goal');
    this.goal.setScale(0.6);

    // enemy group
    this.enemies = this.add.group({
        key: 'enemy',
        repeat: 5,
        setXY: {
            x: 90,
            y: 100,
            stepX: 80,
            stepY: 20
        }
    });

    // setting scale to all group elements
    Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);

    // set flipX, and speed
    Phaser.Actions.Call(this.enemies.getChildren(), function(enemy){
        // flip enemy
        enemy.flipX = true;

        // set speed
        let dir = Math.random() < 0.5 ? 1 : -1;
        let speed = this.enemyMinSpeed + Math.random() * (this.enemyMaxSpeed - this.enemy
```

```
MinSpeed);
    enemy.Speed = dir * speed;

    }, this);
};

// this is called up to 60 times per second
gameScene.update = function(){

    // don't execute if we are terminating
    if(this.isTerminating) return;

    // check for active input (left click / touch)
    if(this.input.activePointer.isDown) {
        // player walks
        this.player.x += this.playerSpeed;
    }

    // treasure overlap check
    let playerRect = this.player.getBounds();
    let treasureRect = this.goal.getBounds();

    if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, treasureRect)) {
        console.log('reached goal!');

        // end game
        return this.gameOver();
    }

    // get enemies
    let enemies = this.enemies.getChildren();
    let numEnemies = enemies.length;

    for(let i = 0; i< numEnemies; i++) {

        // enemy movement
        enemies[i].y += enemies[i].speed;

        // check we haven't passed min or max Y
        let conditionUp = enemies[i].speed < 0 && enemies[i].y <= this.enemyMinY;
        let conditionDown = enemies[i].speed > 0 && enemies[i].y >= this.enemyMaxY;

        // if we passed the upper or lower limit, reverse
        if(conditionUp || conditionDown) {
            enemies[i].speed *= -1;
        }

        // check enemy overlap
        let enemyRect = enemies[i].getBounds();

        if(Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
            console.log('Game over!');

            // end game
            return this.gameOver();
        }
    }
}
```

```
    }  
  }  
};  
  
gameScene.gameOver = function() {  
  
  // initiated game over sequence  
  this.isTerminating = true;  
  
  // shake camera  
  this.cameras.main.shake(500);  
  
  // listen for event completion  
  this.cameras.main.on('camerashakecomplete', function(camera, effect){  
  
    // fade out  
    this.cameras.main.fade(500);  
  }, this);  
  
  this.cameras.main.on('camerafadeoutcomplete', function(camera, effect){  
    // restart the Scene  
    this.scene.restart();  
  }, this);  
  
};  
  
// set the configuration of the game  
let config = {  
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas  
  width: 640,  
  height: 360,  
  scene: gameScene  
};  
  
// create a new game, pass the configuration  
let game = new Phaser.Game(config);
```